

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống bản đồ ẩm thực kết hợp chatbot
gợi ý quán ăn.**

NGUYỄN NHẬT LINH

linh.nn210522@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin: Khoa học máy tính

Giảng viên hướng dẫn: TS. Trịnh Tuấn Đạt

Chữ kí GVHD

Khoa:

Khoa học máy tính

Trường:

Công nghệ Thông tin và Truyền thông

HÀ NỘI, 01/2026

LỜI CẢM ƠN

Lời cảm ơn của sinh viên (SV) tới người yêu, gia đình, bạn bè, thầy cô, và chính bản thân mình vì đã chăm chỉ và quyết tâm thực hiện ĐATN để đạt kết quả tốt nhất, nên viết phần cảm ơn ngắn gọn, tránh dùng các từ sáo rỗng, giới hạn trong khoảng 100-150 từ.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Sinh viên viết tóm tắt ĐATN của mình trong mục này, với 200 đến 350 từ. Theo trình tự, các nội dung tóm tắt cần có: (i) Giới thiệu vấn đề (tại sao có vấn đề đó, hiện tại được giải quyết chưa, có những hướng tiếp cận nào, các hướng này giải quyết như thế nào, hạn chế là gì), (ii) Hướng tiếp cận sinh viên lựa chọn là gì, vì sao chọn hướng đó, (iii) Tổng quan giải pháp của sinh viên theo hướng tiếp cận đã chọn, và (iv) Đóng góp chính của ĐATN là gì, kết quả đạt được sau cùng là gì. Sinh viên cần viết thành đoạn văn, không được viết ý hoặc gạch đầu dòng.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

Mục này khuyến khích sinh viên viết lại mục “Tóm tắt” đồ án tốt nghiệp ở trang trước bằng tiếng Anh. Phần này phải có đầy đủ các nội dung như trong phần tóm tắt bằng tiếng Việt. Sinh viên không nhất thiết phải trình bày mục này.

Nhưng nếu lựa chọn trình bày, sinh viên cần đảm bảo câu từ và ngữ pháp chuẩn tắc, nếu không sẽ có tác dụng ngược, gây phản cảm.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.2 Tổng quan chức năng	5
2.2.1 Biểu đồ use case tổng quan	5
2.2.2 Biểu đồ use case phân rã tương tác với Chatbot.....	7
2.2.3 Biểu đồ use case phân rã tương tác với quán ăn.....	8
2.2.4 Biểu đồ use case phân rã quản lý hồ sơ cá nhân.....	9
2.2.5 Biểu đồ use case phân rã quản lý quán ăn	9
2.2.6 Biểu đồ use case phân rã quản lý người dùng	10
2.2.7 Biểu đồ use case phân rã quản lý đánh giá	10
2.2.8 Quy trình nghiệp vụ	11
2.3 Đặc tả chức năng	13
2.3.1 Đặc tả use case: Tương tác với Chatbot	13
2.3.2 Đặc tả use case: Tương tác với quán ăn	14
2.3.3 Đặc tả use case: Chỉnh sửa hồ sơ cá nhân	15
2.3.4 Đặc tả use case: Quản lý quán ăn.....	15
2.4 Yêu cầu phi chức năng	16
2.4.1 Hiệu năng.....	16
2.4.2 Độ tin cậy.....	16

2.4.3 Tính dễ dùng	16
2.4.4 Tính bảo mật	16
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	17
3.1 Công nghệ phát triển giao diện người dùng	17
3.1.1 ReactJS	17
3.1.2 React-Leaflet.....	17
3.1.3 Axios	18
3.2 Công nghệ phát triển phía máy chủ.....	18
3.2.1 Node.js và Express.js.....	18
3.2.2 PostgreSQL	18
3.2.3 Sequelize ORM	19
3.2.4 JWT và bcryptjs.....	19
3.3 Trí tuệ nhân tạo và Xử lý ngôn ngữ tự nhiên	19
3.3.1 Google Gemini API	20
3.3.2 OpenAI API	20
3.3.3 OpenStreetMap API	20
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	22
4.1 Thiết kế kiến trúc.....	22
4.1.1 Lựa chọn kiến trúc phần mềm	22
4.1.2 Thiết kế tổng quan.....	23
4.1.3 Thiết kế chi tiết gói	25
4.2 Thiết kế chi tiết.....	31
4.2.1 Thiết kế giao diện	31
4.2.2 Thiết kế lớp	34
4.2.3 Thiết kế cơ sở dữ liệu	38

4.3 Xây dựng ứng dụng.....	41
4.3.1 Thư viện và công cụ sử dụng.....	41
4.3.2 Kết quả đạt được	41
4.3.3 Minh họa các chức năng chính	42
4.4 Kiểm thử.....	43
4.4.1 Chiến lược kiểm thử	43
4.4.2 Kiểm thử chức năng xác thực	43
4.4.3 Kiểm thử chức năng quản lý quán ăn.....	44
4.4.4 Kiểm thử chức năng chatbot.....	44
4.4.5 Kiểm thử chức năng đánh giá và yêu thích.....	45
4.4.6 Kiểm thử chức năng gợi ý cá nhân hóa.....	45
4.4.7 Kiểm thử chức năng quản trị	46
4.4.8 Kiểm thử giao diện người dùng	46
4.4.9 Kiểm thử hiệu năng	46
4.4.10 Tổng hợp kết quả kiểm thử	47
4.4.11 Kết luận	47
4.5 Triển khai	48
4.5.1 Mô hình triển khai.....	48
4.5.2 Cấu hình triển khai.....	48
4.5.3 Kết quả triển khai thử nghiệm	48
4.5.4 Hướng dẫn triển khai production	48
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	49
5.1 Giải pháp hệ thống tự học từ cơ sở dữ liệu.....	49
5.1.1 Đặt vấn đề	49
5.1.2 Giải pháp	49
5.1.3 Kết quả đạt được	49

5.2 Cơ chế gợi ý cá nhân hóa dựa trên hồ sơ người dùng	50
5.2.1 Đặt vấn đề	50
5.2.2 Giải pháp	50
5.2.3 Kết quả đạt được	50
5.3 Kiến trúc Chatbot đa mô hình với cơ chế dự phòng	50
5.3.1 Đặt vấn đề	50
5.3.2 Giải pháp	50
5.3.3 Kết quả đạt được	51
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	52
6.1 Kết luận	52
6.2 Hướng phát triển.....	52
TÀI LIỆU THAM KHẢO.....	55

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ Use case tổng quan	6
Hình 2.2	Biểu đồ Use case phân rã tương tác với Chatbot	7
Hình 2.3	Biểu đồ Use case phân rã tương tác với quán ăn	8
Hình 2.4	Biểu đồ Use case phân rã quản lý hồ sơ cá nhân	9
Hình 2.5	Biểu đồ Use case phân rã quản lý quán ăn	9
Hình 2.6	Biểu đồ Use case phân rã quản lý người dùng	10
Hình 2.7	Biểu đồ Use case phân rã quản lý đánh giá	10
Hình 2.8	Biểu đồ Quy trình nghiệp vụ Thêm quán ăn vào yêu thích . .	11
Hình 4.1	Biểu đồ phụ thuộc gói UML	24
Hình 4.2	Thiết kế gói Frontend	27
Hình 4.3	Thiết kế gói Backend	29
Hình 4.4	Thiết kế gói Database	31
Hình 4.5	Biểu đồ thiết kế giao diện trang chủ	33
Hình 4.6	Biểu đồ thiết kế giao diện Danh sách quán ăn	33
Hình 4.7	Biểu đồ thiết kế giao diện Thêm quán ăn	34
Hình 4.8	Biểu đồ trình tự - Sử dụng chatbot tìm quán ăn	36
Hình 4.9	Biểu đồ trình tự - Thêm quán ăn yêu thích	37
Hình 4.10	Biểu đồ thực thể liên kết (E-R Diagram)	39

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả use case Tương tác với Chatbot	13
Bảng 2.2	Đặc tả use case Tương tác với quán ăn	14
Bảng 2.3	Đặc tả use case Chỉnh sửa hồ sơ cá nhân	15
Bảng 2.4	Đặc tả use case Chỉnh sửa quán ăn	15
Bảng 4.1	Danh sách thư viện và công cụ sử dụng	41
Bảng 4.2	Thống kê mã nguồn	42
Bảng 4.3	Test cases cho chức năng xác thực	43
Bảng 4.4	Test cases cho chức năng quản lý quán ăn	44
Bảng 4.5	Test cases cho chức năng chatbot	44
Bảng 4.6	Test cases cho chức năng đánh giá và yêu thích	45
Bảng 4.7	Test cases cho chức năng gợi ý cá nhân hóa	45
Bảng 4.8	Test cases cho chức năng quản trị	46
Bảng 4.9	Test cases cho giao diện người dùng	46
Bảng 4.10	Test cases cho hiệu năng	47
Bảng 4.11	Tổng hợp số lượng test case đã thực hiện	47

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Ẩm thực đóng vai trò quan trọng trong đời sống xã hội, không chỉ đáp ứng nhu cầu ăn uống của cơ thể mà còn trở thành một yếu tố quan trọng trong văn hóa và giao tiếp của con người. Trong bối cảnh đô thị hóa và xã hội phát triển theo hướng hiện đại hóa, nhu cầu tìm kiếm và trải nghiệm ẩm thực phù hợp với hoàn cảnh ngày càng trở nên phổ biến và hiện diện xuyên suốt mọi lứa tuổi và tầng lớp xã hội. Trong thực tế, nhu cầu tìm kiếm địa điểm ẩm thực phù hợp xuất hiện trong nhiều tình huống khác nhau như: nhóm sinh viên muốn thư giãn sau giờ học, các cặp đôi mong muốn có không gian lãng mạn để hâm nóng tình cảm, hay các bậc phụ huynh tìm địa điểm tổ chức sinh nhật cho con. Việc lựa chọn đúng địa điểm không chỉ đáp ứng nhu cầu ăn uống mà còn góp phần tạo nên những trải nghiệm và kỷ niệm đáng nhớ.

Nhằm đáp ứng những yêu cầu đa dạng này, các quán ăn xuất hiện ngày càng nhiều, với đa dạng món ăn, chất lượng, giá cả, dịch vụ,... Tuy nhiên, thực tế chỉ ra rằng số lượng quán ăn tăng tương ứng với thời gian người dùng bỏ ra để lựa chọn quán ăn phù hợp cũng tăng theo. Hơn nữa, thông tin về quán ăn thường được người dùng tiếp cận từ nhiều nguồn khác nhau bao gồm các trang mạng xã hội như Facebook, TikTok; các ứng dụng tìm kiếm như Google, Cốc Cốc; các ứng dụng ẩm thực như Foody, ShopeeFood. Với lượng thông tin đồ sộ cùng với các nguồn thông tin phân tán, không đồng nhất và được cập nhật đầy đủ, thường xuyên, nhiều người thường phải trả lời cho những câu hỏi không hề dễ dàng như hôm nay ăn gì, ở đâu, quán nào ngon hơn, đảm bảo chất lượng, nhất là vào những dịp đặc biệt như sinh nhật, hẹn hò.

Do đó, vấn đề được đặt ra là cần có một nền tảng tập trung, vừa cung cấp bản đồ trực quan, vừa hỗ trợ gợi ý một cách thông minh dựa trên nhu cầu, ngữ cảnh và sở thích cá nhân của người dùng. Nếu được giải quyết, nền tảng này có thể tiết kiệm thời gian cho người dùng, nâng cao chất lượng trải nghiệm ẩm thực cho tất cả khách hàng tiềm năng. Hơn nữa mô hình này có thể được mở rộng áp dụng cho các lĩnh vực khác như thời trang, giải trí, dịch vụ.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay, để tìm kiếm quán ăn, người dùng có thể sử dụng ứng dụng ẩm thực đặc thù như Foody, ShopeeFood hoặc sử dụng ứng dụng tìm kiếm tổng quát kết hợp bản đồ như Google và Google Maps. Các ứng dụng này đã cung cấp nhiều tính năng khác nhau như thực đơn chính kèm theo hình ảnh món ăn và đánh giá từ

cộng đồng và khách hàng trước. Ngoài ra, khách hàng còn được cung cấp bản đồ chỉ đường đến địa điểm của quán hay thông tin liên lạc trực tiếp đến quán ăn.

Tuy nhiên, vẫn còn một số vấn đề mà những giải pháp được nêu trên gặp phải. Thứ nhất, thông tin chủ yếu ở dạng tra cứu thủ công, yêu cầu người dùng nhập từ khóa và tìm kiếm, so sánh, chọn lọc giữa hàng chục, hàng trăm kết quả trả về. Điều này lấy đi nhiều thời gian và công sức của người dùng cho đôi khi chỉ là một bữa ăn tối thông thường. Thứ hai, người dùng cũng phải có yêu cầu cụ thể về lựa chọn của mình như món ăn chính để cho ra kết quả tốt nhất. Với những yêu cầu không rõ ràng như hẹn hò, tổ chức sinh nhật thì các ứng dụng thường đưa ra kết quả không phù hợp, đặc biệt với những khách hàng có yêu cầu mơ hồ, chưa có diễn đạt cụ thể.

Từ những hạn chế được nêu trên, đề tài hướng tới mục tiêu xây dựng một ứng dụng web bản đồ ẩm thực đơn giản, thuận tiện. Ứng dụng sẽ tích hợp chatbot để trao đổi và đưa ra gợi ý về món ăn và địa điểm nhằm đáp ứng yêu cầu của khách hàng. Từ đó, ứng dụng mong muốn có thể đem lại trải nghiệm tìm kiếm và lựa chọn địa điểm ẩm thực nhanh chóng, thuận lợi và có tính cá nhân hóa cao.

1.3 Định hướng giải pháp

Dựa trên mục tiêu đã đề ra, đề tài định hướng giải quyết bài toán theo hướng kết hợp công nghệ bản đồ trực tuyến với kỹ thuật xử lý ngôn ngữ tự nhiên - Natural Language Processing (NLP).

Bước đầu tiên là thu thập dữ liệu về các địa điểm ẩm thực (tên quán, loại món ăn, địa chỉ, giá trung bình, đánh giá, hình ảnh, v.v.) từ các nguồn công khai hoặc dữ liệu tự tạo, đồng thời xây dựng tập dữ liệu mẫu cho chatbot (bao gồm các câu hỏi, ý định, từ khóa và phản hồi tương ứng), dữ liệu sau khi thu thập sẽ được chuẩn hóa và lưu trữ trong cơ sở dữ liệu quan hệ. Tiếp theo, các chức năng chính của hệ thống được xác định gồm tìm kiếm địa điểm, gợi ý món ăn theo dịp, trò chuyện với chatbot, định vị và dẫn đường.

Đồ án này tích hợp API bản đồ để hiển thị các địa điểm ăn uống, cho phép người dùng tìm kiếm, lọc theo tiêu chí (loại món ăn, khoảng cách, giá tiền, đánh giá, ...) và dẫn đường đến địa điểm được chọn. Bên cạnh đó, đồ án ứng dụng các mô hình xử lý ngôn ngữ tự nhiên (NLP) để nhận diện ý định (intent recognition) và trích xuất thực thể (entity extraction) từ truy vấn của người dùng. Hệ thống chatbot sẽ phân tích dữ liệu đầu vào để đưa ra các phản hồi và gợi ý chính xác dựa trên cơ sở dữ liệu có sẵn. Toàn bộ các thành phần gồm bản đồ, chatbot và hệ quản trị dữ liệu được tích hợp đồng bộ trên nền tảng Web thông qua stack công nghệ ReactJS, Node.js và PostgreSQL, nhằm tối ưu hóa hiệu năng và cung cấp giao diện thân thiện, nhất quán.

Cuối cùng là bước thực hiện kiểm thử hệ thống (chức năng, hiệu năng, giao diện, khả năng phản hồi của chatbot) và đánh giá hiệu quả thông qua phản hồi người dùng thử nghiệm. Kết quả thu thập được sẽ được dùng để điều chỉnh và hoàn thiện ứng dụng.

Những đóng góp chính của đề án bao gồm việc xây dựng một ứng dụng web bản đồ ẩm thực tích hợp chatbot AI có khả năng gợi ý địa điểm và món ăn theo nhu cầu người dùng, mang lại trải nghiệm tương tác tự nhiên và thuận tiện hơn so với các ứng dụng tra cứu truyền thống. Trong quá trình phát triển, đề án đã giải quyết các thách thức về thu thập dữ liệu quán ăn, xử lý ngôn ngữ tự nhiên tiếng Việt và tích hợp bản đồ với hệ thống chatbot. Nhờ quá trình thử nghiệm và tinh chỉnh liên tục, hệ thống đạt được độ ổn định cao, giao diện trực quan và khả năng mở rộng, có thể áp dụng cho các lĩnh vực khác như du lịch, giải trí hoặc gợi ý dịch vụ địa điểm.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày chi tiết về khảo sát hiện trạng các hệ thống hỗ trợ tìm kiếm ẩm thực hiện có, đồng thời phân tích các yêu cầu chức năng và phi chức năng của hệ thống. Trong chương này, các biểu đồ use case tổng quan và phân rã cũng được xây dựng để định hình rõ ràng các nghiệp vụ chính như tìm kiếm trên bản đồ, tương tác với chatbot và quản lý hồ sơ cá nhân của người dùng.

Chương 3 giới thiệu về các công nghệ cốt lõi được sử dụng trong đề án bao gồm ReactJS, Node.js và hệ quản trị cơ sở dữ liệu PostgreSQL. Nội dung trọng tâm của chương là việc tìm hiểu và ứng dụng Google Gemini API trong xử lý ngôn ngữ tự nhiên, cùng với các thư viện hỗ trợ bản đồ trực tuyến như Leaflet để xây dựng giao diện tương tác trực quan.

Chương 4 tập trung vào quá trình thiết kế và triển khai chi tiết ứng dụng. Chương này mô tả kiến trúc phần mềm, thiết kế cơ sở dữ liệu với 8 bảng quan hệ và đặc tả các thành phần giao diện quan trọng. Đồng thời, quy trình kiểm thử hệ thống cũng được trình bày để đảm bảo tính ổn định của các tính năng và khả năng phản hồi chính xác của chatbot.

Chương 5 trình bày các giải pháp và đóng góp nổi bật nhất của đề án, đặc biệt là hệ thống tự học từ cơ sở dữ liệu và thuật toán gợi ý cá nhân hóa dựa trên hồ sơ người dùng. Đây là phần trọng tâm thể hiện khả năng sáng tạo và giải quyết vấn đề của sinh viên trong việc tối ưu hóa trải nghiệm tìm kiếm ẩm thực thông qua trí tuệ nhân tạo.

Chương 6 đưa ra những kết luận tổng kết về kết quả đạt được sau quá trình thực

hiện đồ án, đồng thời phân tích những hạn chế còn tồn tại. Trên cơ sở đó, chương cũng đề xuất các hướng phát triển tiềm năng trong tương lai như cải thiện khả năng NLP tiếng Việt chuyên sâu hay mở rộng ứng dụng lên nền tảng di động.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã thảo luận về mục tiêu và phạm vi của đề tài xây dựng hệ thống bản đồ ẩm thực kết hợp trí tuệ nhân tạo. Chương 2 này sẽ trình bày chi tiết về quá trình khảo sát hiện trạng, phân tích các chức năng cần thiết và đưa ra các đặc tả yêu cầu chi tiết cho hệ thống. Nội dung chính của chương bao gồm: (i) Khảo sát hiện trạng và các ứng dụng tương tự trong phần 2.1, (ii) Tổng quan các chức năng hệ thống trong phần 2.2, (iii) Đặc tả chi tiết các Use Case quan trọng trong phần 2.3, và (iv) Các yêu cầu phi chức năng trong phần 2.4.

2.1 Khảo sát hiện trạng

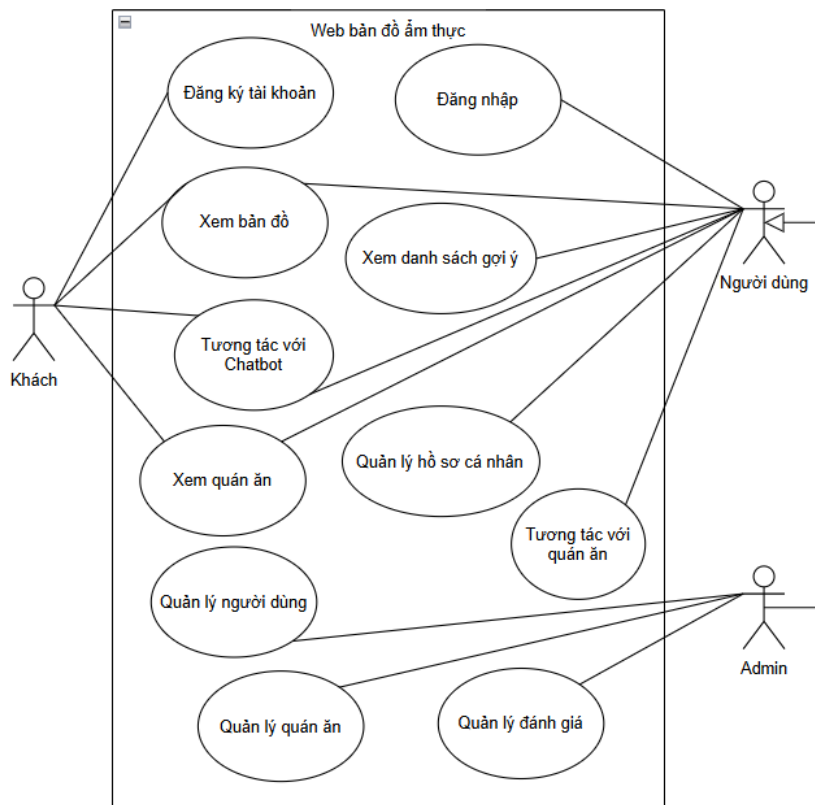
Hiện nay, các dịch vụ bản đồ như Google Maps hay các nền tảng ẩm thực như Foody, ShopeeFood đã cung cấp khả năng tìm kiếm quán ăn khá mạnh mẽ. Tuy nhiên, các hệ thống này vẫn tồn tại một số hạn chế: (i) Lượng thông tin quá lớn khiến người dùng khó đưa ra quyết định, (ii) Thiếu sự tương tác tự nhiên thông qua ngôn ngữ, và (iii) Khả năng cá nhân hóa gợi ý dựa trên hồ sơ người dùng chưa thực sự sâu sắc.

Hệ thống được đề xuất hướng tới việc khắc phục những hạn chế này bằng cách kết hợp bản đồ tương tác với Chatbot AI thông minh. Thay vì chỉ hiển thị danh sách tĩnh, hệ thống sẽ sử dụng công nghệ xử lý ngôn ngữ tự nhiên (NLP) để tư vấn và gợi ý quán ăn phù hợp nhất với nhu cầu tức thời của người dùng.

2.2 Tổng quan chức năng

2.2.1 Biểu đồ use case tổng quan

Biểu đồ use case tổng quan trong Hình 2.1 minh họa phạm vi hoạt động của hệ thống và mối tương quan giữa các tác nhân với các chức năng chính.

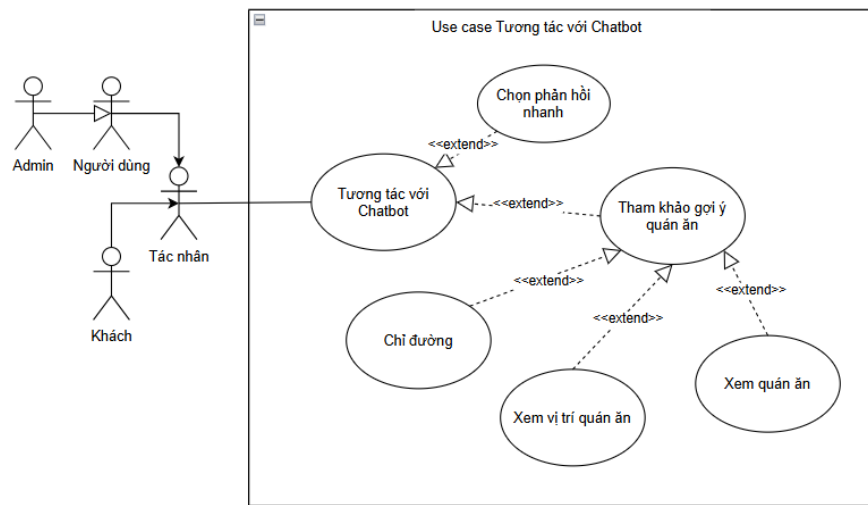


Hình 2.1: Biểu đồ Use case tổng quan

Trong mô hình này, tác nhân Khách là khách hàng tiềm năng của ứng dụng. Họ có thể sử dụng một số tính năng của web như xem bản đồ, tương tác với Chatbot và xem các quán ăn được Chatbot gợi ý. Ngoài ra họ có thể sử dụng tính năng đăng ký tài khoản và trở thành tác nhân Người dùng. Người dùng sẽ có thể đăng nhập vào tài khoản của mình, thiết lập và quản lý hồ sơ cá nhân và xem các danh sách gợi ý được cá nhân hóa. Thêm vào đó, họ cũng có thể tương tác với những quán ăn bằng cách đưa đánh giá, nhận xét, yêu thích hay chặn một quán ăn nhất định.

Tác nhân thứ ba là Admin - Quản trị viên, giữ vai trò điều phối và giám sát hoạt động của ứng dụng. Tương ứng với vai trò đó, Admin có thể thực hiện những tính năng cao cấp bao gồm quản lý quán ăn để cập nhật những thông tin mới nhất về các quán ăn như giờ mở - đóng cửa, giá cả, tiện ích, v.v.. ; quản lý danh sách người dùng và quản lý đánh giá của người dùng cho các quán ăn để đảm bảo tính xác thực và giải quyết những vấn đề phát sinh. Admin cũng được coi như một Người dùng và được phép sử dụng các tính năng Người dùng có thể thực hiện.

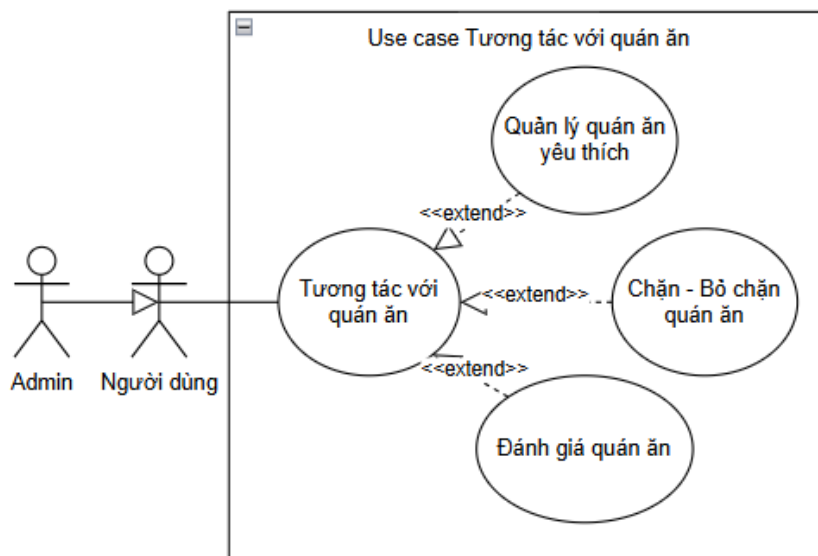
2.2.2 Biểu đồ use case phân rã tương tác với Chatbot



Hình 2.2: Biểu đồ Use case phân rã tương tác với Chatbot

Với use case tương tác với Chatbot, các tác nhân Người dùng, Khách hay Admin sẽ được gọi chung là Tác nhân. Nếu lịch sử trò chuyện với Chatbot trống, Tác nhân có thể chọn một số phản hồi mẫu cho câu hỏi của Chatbot về loại quán ăn họ muốn được gợi ý. Nếu Tác nhân có những yêu cầu cụ thể hơn, họ có thể nêu chúng ra khi trò chuyện để Chatbot lọc và tìm kiếm danh sách những quán ăn phù hợp và gợi ý chúng cho Tác nhân. Khi nhận được danh sách, Tác nhân có thể xem chi tiết hơn từng quán ăn, xem vị trí của chúng trên bản đồ và nếu lựa chọn được quán ăn đúng theo mong muốn của mình, họ có thể sử dụng tính năng Chỉ đường tới quán từ vị trí hiện tại.

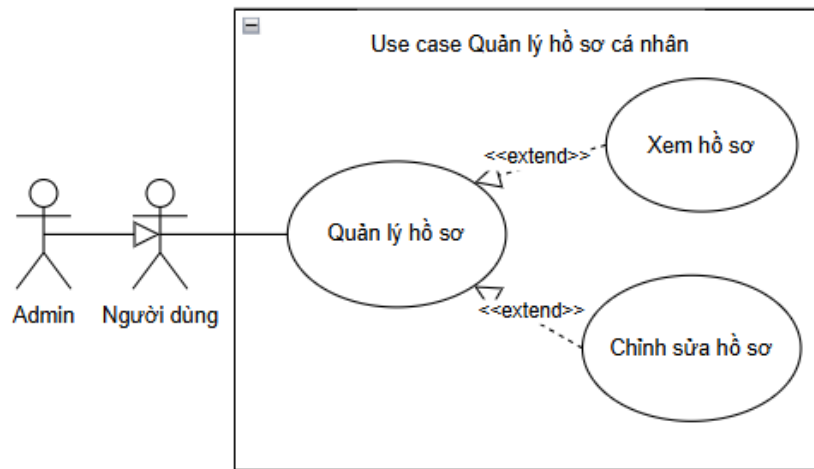
2.2.3 Biểu đồ use case phân rã tương tác với quán ăn



Hình 2.3: Biểu đồ Use case phân rã tương tác với quán ăn

Những tác nhân đã tạo tài khoản cho trang web bao gồm Người dùng và Admin sẽ được sử dụng Use case tương tác với quán ăn. Cụ thể hơn, trang thông tin chi tiết của từng quán ăn có ba nút tương tác với quán ăn: Yêu thích, Chặn và Viết đánh giá. Nút Yêu thích sẽ giúp Người dùng đưa quán vào danh sách yêu thích của mình, danh sách đó có thể được truy cập trên một trang riêng biệt để quản lý. Nút Chặn hỗ trợ Chatbot bỏ qua quán bị chặn khi gợi ý quán ăn cho Người dùng. Mặt khác, Người dùng cũng có thể bỏ yêu thích hay bỏ chặn quán ăn khi cần. Nếu Người dùng muốn đánh giá quán ăn để cho những người dùng sau tham khảo, họ có thể sử dụng nút Viết đánh giá để đánh giá quán ăn theo thang từ 1 đến 5 sao và viết nhận xét cụ thể hơn nếu có.

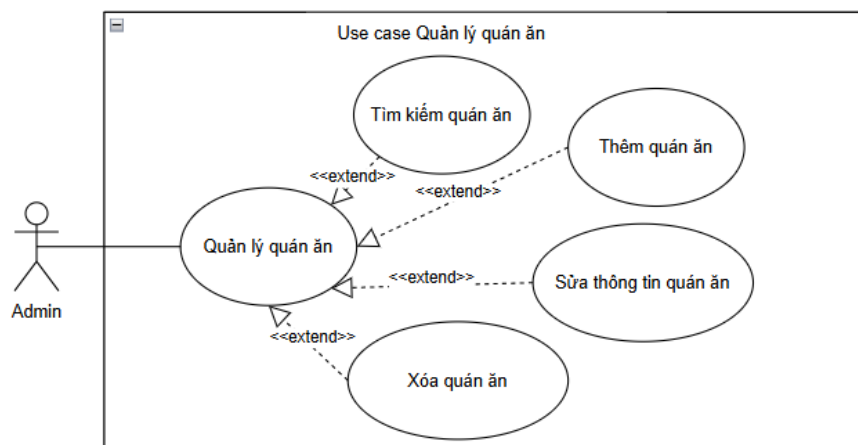
2.2.4 Biểu đồ use case phân rã quản lý hồ sơ cá nhân



Hình 2.4: Biểu đồ Use case phân rã quản lý hồ sơ cá nhân

Mỗi Người dùng có thể truy cập vào trang Hồ sơ để xem lại thông tin cá nhân, món ăn ưa thích, dị ứng, v.v... Nếu có sự thay đổi, họ có thể chỉnh sửa lại hồ sơ theo cập nhật mới nhất của mình.

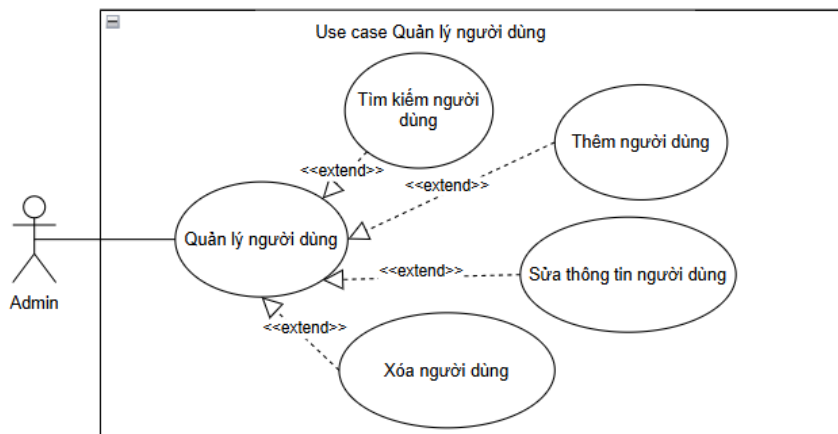
2.2.5 Biểu đồ use case phân rã quản lý quán ăn



Hình 2.5: Biểu đồ Use case phân rã quản lý quán ăn

Admin có thể truy cập vào trang Quản lý quán ăn từ trang chủ của ứng dụng web qua một nút đặc biệt Admin. Tại đó Admin có thể tìm kiếm quán ăn theo tên, thêm quán ăn mới và nhập các trường thông tin tương ứng, chỉnh sửa thông tin quán ăn nếu có sai sót hoặc cập nhật mới, hay xóa quán ăn đã đóng cửa trong danh sách.

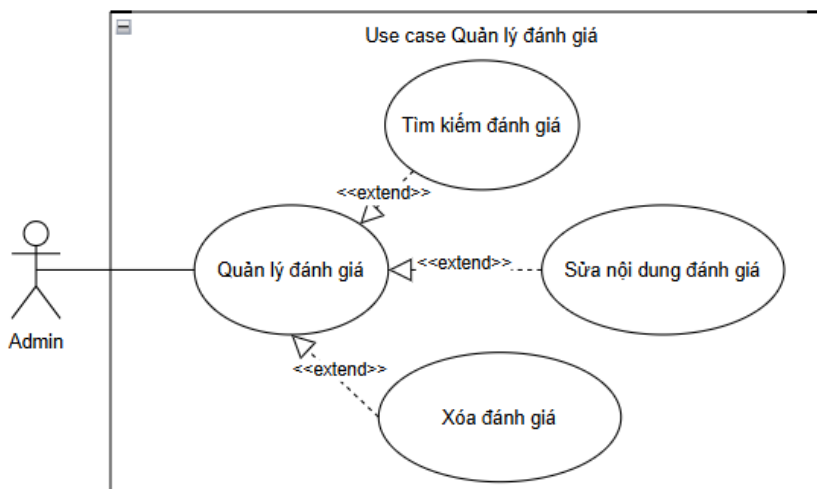
2.2.6 Biểu đồ use case phân rã quản lý người dùng



Hình 2.6: Biểu đồ Use case phân rã quản lý người dùng

Từ trang Quản lý quán ăn, Admin của ứng dụng có thể truy cập vào trang Quản lý người dùng để xem danh sách người dùng; tìm kiếm người dùng theo tên, username, email; thêm người dùng mới mà không cần thông qua trang Đăng ký tài khoản; chỉnh sửa thông tin nếu cần thiết, hoặc xóa người dùng khỏi trang web.

2.2.7 Biểu đồ use case phân rã quản lý đánh giá



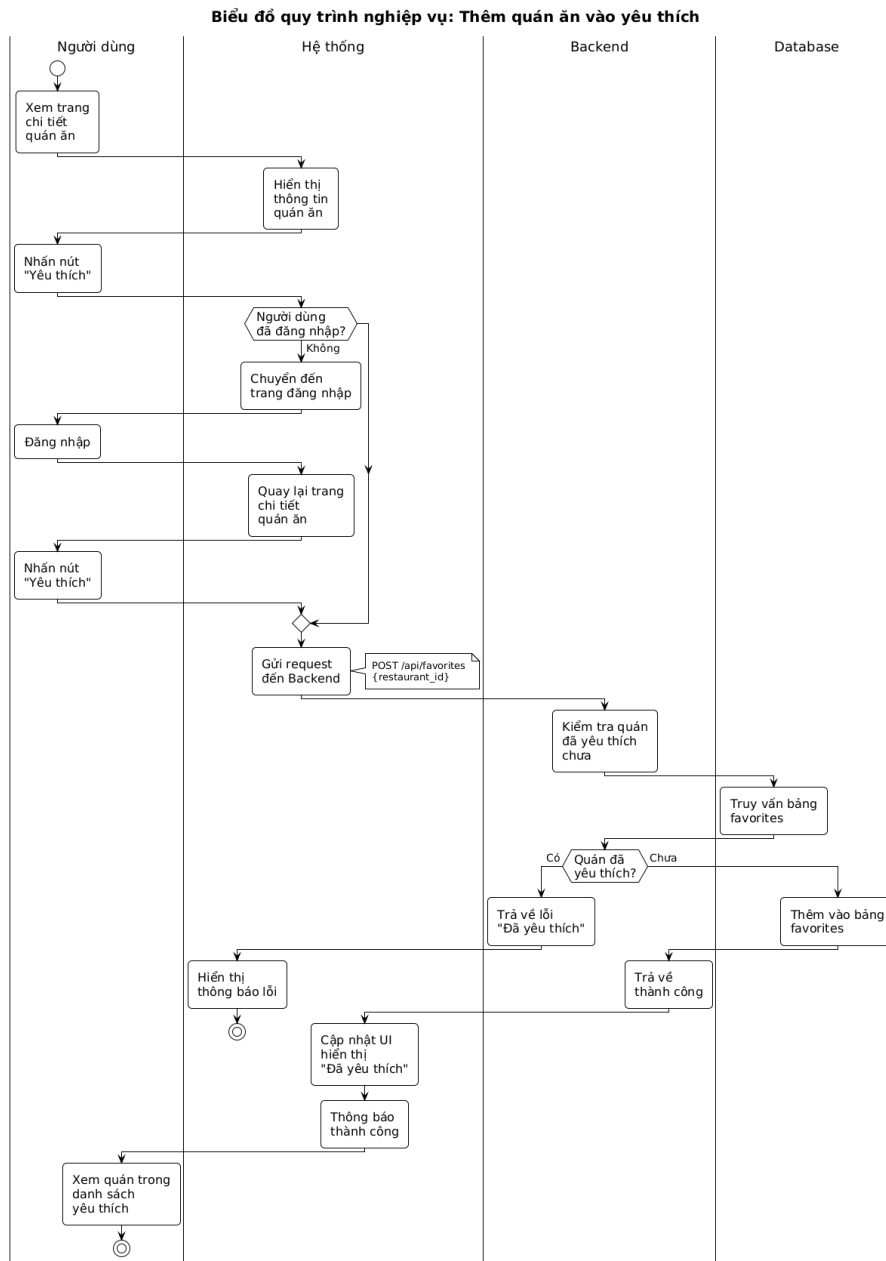
Hình 2.7: Biểu đồ Use case phân rã quản lý đánh giá

Cũng từ trang Quản lý quán ăn, Admin của ứng dụng có thể truy cập vào trang Quản lý đánh giá để xem danh sách đánh giá; tìm kiếm đánh giá theo nội dung, quán ăn, người dùng; chỉnh sửa hoặc xác minh nội dung đánh giá; và xóa đánh giá ảo, xấu hay sử dụng ngôn từ không phù hợp.

2.2.8 Quy trình nghiệp vụ

Quy trình nghiệp vụ Thêm quán ăn vào yêu thích là một quy trình đơn giản nhưng quan trọng trong hệ thống, cho phép người dùng lưu lại các quán ăn mà họ quan tâm để xem lại sau này. Quy trình này thể hiện sự tương tác giữa các thành phần của hệ thống bao gồm người dùng, Frontend, Backend và Database. Đồ án đã thiết kế quy trình này với các bước xử lý logic, tuần tự, đảm bảo tính nhất quán dữ liệu và trải nghiệm người dùng tốt.

Biểu đồ quy trình nghiệp vụ được trình bày trong Hình 2.8. Biểu đồ này mô tả chi tiết các bước xử lý từ khi người dùng xem trang chi tiết quán ăn đến khi quán ăn được thêm vào danh sách yêu thích, bao gồm các điều kiện kiểm tra và xử lý lỗi.



Hình 2.8: Biểu đồ Quy trình nghiệp vụ Thêm quán ăn vào yêu thích

Quy trình bắt đầu khi người dùng xem trang chi tiết quán ăn. Khi người dùng truy cập trang chi tiết quán ăn, hệ thống Frontend sẽ hiển thị đầy đủ thông tin về quán ăn bao gồm tên quán, địa chỉ, đánh giá, mô tả, hình ảnh, giờ mở cửa, và nút Yêu thích. Nếu quán ăn chưa được yêu thích, nút sẽ hiển thị Yêu thích, còn nếu đã được yêu thích, nút sẽ hiển thị Đã yêu thích. Khi người dùng nhấn vào nút Yêu thích trên giao diện, hệ thống Frontend sẽ nhận sự kiện click và bắt đầu xử lý yêu cầu. Hệ thống sẽ kiểm tra trạng thái đăng nhập của người dùng để xác định xem người dùng có quyền thực hiện hành động này hay không. Nếu người dùng đã đăng nhập, hệ thống sẽ tiếp tục đến bước tiếp theo. Nếu người dùng chưa đăng nhập, hệ thống sẽ tự động chuyển hướng đến trang đăng nhập. Sau khi người dùng thực hiện đăng nhập thành công, hệ thống sẽ quay lại trang chi tiết quán ăn và người dùng có thể nhấn lại nút Yêu thích để tiếp tục quy trình.

Sau khi xác nhận người dùng đã đăng nhập, Frontend sẽ gọi API POST `/api/favorites` với dữ liệu gửi kèm bao gồm `restaurant_id` là ID của quán ăn và `token` xác thực được lấy từ `localStorage`. Backend nhận được yêu cầu và bắt đầu xử lý bằng cách truy vấn bảng `favorites` trong Database để kiểm tra xem đã tồn tại bản ghi với `user_id` và `restaurant_id` tương ứng chưa. Nếu quán đã được yêu thích trước đó, Backend sẽ trả về lỗi Đã yêu thích, Frontend sẽ hiển thị thông báo lỗi cho người dùng và quy trình kết thúc. Nếu quán chưa được yêu thích, Backend sẽ tạo bản ghi mới trong bảng `favorites` với các trường dữ liệu bao gồm `user_id` là ID người dùng, `restaurant_id` là ID quán ăn, và `createdAt` là thời gian tạo. Sau khi thêm thành công, Backend trả về response thành công cho Frontend.

Frontend nhận response thành công từ Backend và cập nhật giao diện người dùng. Nút Yêu thích được đổi thành Đã yêu thích với màu sắc hoặc icon khác để thể hiện trạng thái mới. Hệ thống cũng hiển thị thông báo thành công cho người dùng để xác nhận hành động đã được thực hiện. Người dùng có thể truy cập trang Yêu thích để xem danh sách các quán ăn đã được yêu thích, và quán ăn vừa thêm sẽ xuất hiện trong danh sách này, xác nhận rằng quy trình đã hoàn tất thành công.

2.3 Đặc tả chức năng

2.3.1 Đặc tả use case: Tương tác với Chatbot

Mã use case	UC-01	Tên use case	Tương tác với Chatbot
Tác nhân	Admin, Người dùng, Khách (Gộp chung thành Tác nhân); Chatbot; Hệ thống.		
Tiền điều kiện	Tác nhân đang ở trang chủ.		
Luồng sự kiện chính	STT	Thực hiện	Hành động
	1	Tác nhân	Nhập yêu cầu hoặc câu hỏi.
	2	Hệ thống	Gửi tin nhắn của người dùng cho chatbot.
	3	Chatbot	Tìm kiếm, lên danh sách các quán ăn phù hợp và tạo câu trả lời.
	4	Hệ thống	Hiển thị câu trả lời, danh sách quán ăn gợi ý và hiển thị vị trí của quán ăn trên bản đồ.
Luồng sự kiện phát sinh	STT	Thực hiện	
	1a	Tác nhân	Chọn một phản hồi nhanh.
	3b-1	Chatbot	Không tìm thấy quán ăn phù hợp.
	3b-2	Chatbot	Không trả về danh sách gợi ý.
	4b	Hệ thống	Không hiển thị vị trí quán ăn trên bản đồ.
Hậu điều kiện	Tác nhân nhận được danh sách các quán ăn gợi ý để tra cứu. Các quán ăn được hiển thị trên bản đồ.		

Bảng 2.1: Đặc tả use case Tương tác với Chatbot

2.3.2 Đặc tả use case: Tương tác với quán ăn

Mã use case	UC-02	Tên use case	Tương tác với quán ăn
Tác nhân	Admin, Người dùng (Gộp chung thành Tác nhân); Hệ thống.		
Tiền điều kiện	Tác nhân đang ở trang Danh sách quán ăn.		
Luồng sự kiện chính	STT	Thực hiện	Hành động
	1	Tác nhân	Lựa chọn "Xem chi tiết" một quán ăn.
	2	Hệ thống	Chuyển tác nhân đến trang của quán ăn được chọn.
	3	Tác nhân	Lựa chọn hành động mong muốn.
	3a	Tác nhân	Chọn / Bỏ chọn Yêu thích quán ăn.
	4a	Hệ thống	Cập nhật thông tin vào cơ sở dữ liệu.
	5a	Hệ thống	Hiển thị trạng thái Đã yêu thích / Yêu thích.
	3b	Tác nhân	Chặn / Bỏ chặn quán ăn.
	4b	Hệ thống	Cập nhật thông tin vào cơ sở dữ liệu.
	5b-1	Hệ thống	Thông báo đã chặn / bỏ chặn thành công.
	5b-2	Hệ thống	Hiển thị trạng thái Đã chặn / Chặn.
	3c	Tác nhân	Chọn "Viết đánh giá"
	4c	Hệ thống	Hiển thị mục "Đánh giá" và "Nhận xét".
	5c	Tác nhân	Đánh giá quán ăn theo số sao và gửi đánh giá.
	6c	Hệ thống	Cập nhật các thông tin tương ứng và thêm đánh giá vào cơ sở dữ liệu.
	7c	Hệ thống	Cập nhật thông tin của quán ăn và hiển thị đánh giá của người dùng.
Luồng sự kiện phát sinh	STT	Thực hiện	
	1b-1	Tác nhân	Tìm kiếm quán ăn theo tiêu chí nhất định (tên, loại món,...)
	1b-2	Hệ thống	Cập nhật danh sách quán ăn hiển thị phù hợp.
	3d	Tác nhân	Chọn "Xem trên bản đồ"
	4d	Hệ thống	Chuyển người dùng sang trang Google Maps hiển thị vị trí quán ăn theo kinh độ và vĩ độ.
	5c-2	Tác nhân	Hủy đánh giá.
	6c-2	Hệ thống	Không hiển thị mục "Đánh giá" và "Nhận xét".
Hậu điều kiện	Những thay đổi được cập nhật trong cơ sở dữ liệu và hiển thị trên trang quán ăn tương ứng.		

Bảng 2.2: Đặc tả use case Tương tác với quán ăn

2.3.3 Đặc tả use case: Chỉnh sửa hồ sơ cá nhân

Mã use case	UC-03	Tên use case	Chỉnh sửa hồ sơ cá nhân
Tác nhân	Admin, Người dùng (Gộp chung thành Tác nhân); Hệ thống.		
Tiền điều kiện	Tác nhân đang ở trang Hồ sơ.		
Luồng sự kiện chính	STT	Thực hiện	Hành động
	1	Tác nhân	Lựa chọn chỉnh sửa hồ sơ.
	2	Hệ thống	Chuyển tác nhân đến trang chỉnh sửa hồ sơ.
	3	Tác nhân	Cập nhật hồ sơ.
	4	Tác nhân	Xác nhận lưu thay đổi.
	5	Hệ thống	Cập nhật thông tin vào cơ sở dữ liệu.
	6	Hệ thống	Hiển thị hồ sơ sau khi chỉnh sửa tại trang Hồ sơ.
Luồng sự kiện phát sinh	STT	Thực hiện	
	4a	Tác nhân	Hủy chỉnh sửa hồ sơ.
	5a	Hệ thống	Chuyển người dùng về trang Hồ sơ.
Hậu điều kiện	Hồ sơ của người dùng được cập nhật trong cơ sở dữ liệu và được hiển thị trên trang Hồ sơ.		

Bảng 2.3: Đặc tả use case Chỉnh sửa hồ sơ cá nhân

2.3.4 Đặc tả use case: Quản lý quán ăn

Mã use case	UC-04	Tên use case	Chỉnh sửa quán ăn
Tác nhân	Admin; Hệ thống.		
Tiền điều kiện	Admin đang ở trang Thay đổi quán ăn.		
Luồng sự kiện chính	STT	Thực hiện	Hành động
	1	Admin	Lựa chọn sửa một quán ăn.
	2	Hệ thống	Chuyển tác nhân đến trang chỉnh sửa thông tin quán ăn được chọn.
	3	Admin	Cập nhật các thông tin mới.
	4	Admin	Xác nhận lưu thay đổi
	5	Hệ thống	Cập nhật thông tin vào cơ sở dữ liệu.
	6	Hệ thống	Thông báo chỉnh sửa quán ăn thành công.
Luồng sự kiện phát sinh	STT	Thực hiện	
	4a	Admin	Hủy thay đổi thông tin quán ăn.
	5a	Hệ thống	Chuyển Admin về trang Thay đổi quán ăn.
Hậu điều kiện	Thông tin quán ăn được cập nhật trong cơ sở dữ liệu. Admin được đưa về trang Thay đổi quán ăn.		

Bảng 2.4: Đặc tả use case Chỉnh sửa quán ăn

2.4 Yêu cầu phi chức năng

2.4.1 Hiệu năng

Về thời gian phản hồi của trang web, thời gian tải trang không vượt quá 3 giây và thời gian phản hồi của Chatbot không vượt quá 10 giây. Hệ thống cũng phải hỗ trợ ít nhất 50 người dùng tại cùng một thời điểm.

2.4.2 Độ tin cậy

Hệ thống phải hoạt động tại ít nhất 95% thời gian. Khi gặp lỗi, hệ thống cần phải xử lý lỗi trong một khoảng thời gian nhất định và thông báo cho người dùng khi không thể khắc phục lỗi trong khoảng thời gian đó.

2.4.3 Tính dễ dùng

Giao diện của web cần hỗ trợ cho máy tính và điện thoại. Màu sắc có độ tương phản vừa đủ, dễ nhìn. Các tính năng chính của hệ thống phải có tính trực quan, rõ ràng cho người dùng mọi lứa tuổi.

2.4.4 Tính bảo mật

Bảo mật các thông tin của người dùng, đặc biệt là hồ sơ cá nhân và email. Người dùng chỉ có thể chỉnh sửa các thông tin trong tài khoản của mình.

Chương 2 đã hoàn thành việc khảo sát hiện trạng và xác định rõ ràng các yêu cầu chức năng thông qua hệ thống đặc tả use case chi tiết. Những phân tích về nhu cầu người dùng và các hạn chế của hệ thống hiện tại là cơ sở để đề án lựa chọn các công nghệ phù hợp trong chương tiếp theo. Chương 3 sẽ trình bày chi tiết về các nền tảng kỹ thuật và thư viện được sử dụng để hiện thực hóa những yêu cầu này.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Chương 2 đã phân tích chi tiết các yêu cầu chức năng và phi chức năng của hệ thống, đặc biệt là nhu cầu về một bản đồ tương tác trực quan và một chatbot có khả năng xử lý ngôn ngữ tự nhiên tốt. Chương 3 này sẽ tập trung giới thiệu và phân tích các công nghệ, nền tảng lý thuyết được lựa chọn để hiện thực hóa các yêu cầu đó. Việc lựa chọn công nghệ phù hợp là yếu tố quyết định đến hiệu năng, khả năng mở rộng và trải nghiệm người dùng của hệ thống. Đồ án đã cân nhắc kỹ lưỡng giữa các yếu tố như hiệu năng, chi phí, cộng đồng hỗ trợ, và khả năng tích hợp để đưa ra các quyết định công nghệ tối ưu. Nội dung chương bao gồm ba phần chính: công nghệ phát triển giao diện người dùng, công nghệ phát triển phía máy chủ và cơ sở dữ liệu, và các giải pháp trí tuệ nhân tạo.

3.1 Công nghệ phát triển giao diện người dùng

Để đáp ứng yêu cầu về giao diện hiện đại, tốc độ phản hồi nhanh và khả năng tương tác mượt mà trên bản đồ, đồ án đã lựa chọn một bộ công nghệ giao diện người dùng hiện đại và mạnh mẽ.

3.1.1 ReactJS

ReactJS là thư viện JavaScript phổ biến được phát triển bởi Facebook để xây dựng giao diện người dùng dựa trên các thành phần. Trong đồ án, ReactJS đóng vai trò là nền tảng chính cho toàn bộ giao diện người dùng, cho phép xây dựng các thành phần tái sử dụng và quản lý trạng thái ứng dụng một cách hiệu quả. ReactJS giúp quản lý trạng thái ứng dụng thông qua cơ chế DOM ảo, cho phép cập nhật giao diện nhanh chóng mà không cần tải lại toàn bộ trang. Điều này đặc biệt quan trọng trong đồ án khi cần cập nhật đồng bộ giữa danh sách quán ăn và các điểm đánh dấu trên bản đồ. ReactJS cũng cung cấp hệ thống hook như `useState` và `useEffect` để quản lý trạng thái cục bộ, và `React Context API` để quản lý trạng thái toàn cục như trạng thái xác thực.

3.1.2 React-Leaflet

React-Leaflet là thư viện quan trọng để triển khai bản đồ tương tác trong đồ án. Đây là lớp bọc React cho thư viện Leaflet, một thư viện JavaScript mã nguồn mở phổ biến để hiển thị bản đồ tương tác. Lựa chọn Leaflet thay vì Google Maps API giúp tiết kiệm chi phí vận hành đáng kể do Leaflet là mã nguồn mở và miễn phí. Leaflet sử dụng dữ liệu bản đồ từ OpenStreetMap, một dự án bản đồ cộng đồng miễn phí và mở. Trong đồ án, React-Leaflet được sử dụng để hiển thị bản đồ Hà Nội với các điểm đánh dấu cho vị trí người dùng và các quán ăn được chatbot gợi

ý.

3.1.3 Axios

Axios là thư viện JavaScript phổ biến để thực hiện các yêu cầu HTTP tới giao diện lập trình ứng dụng phía máy chủ một cách tin cậy. Trong đồ án, Axios đóng vai trò là cầu nối giữa giao diện người dùng và phía máy chủ, cho phép các thành phần React gọi giao diện lập trình ứng dụng để lấy dữ liệu hoặc thực hiện các thao tác như tạo, cập nhật, xóa dữ liệu. Axios được cấu hình với bộ chặn để tự động thêm mã thông báo JWT vào tiêu đề của mọi yêu cầu khi người dùng đã đăng nhập, và được cấu hình với thời gian chờ 30 giây để tránh các yêu cầu bị treo quá lâu.

3.2 Công nghệ phát triển phía máy chủ

Phía máy chủ cần xử lý các tác vụ nặng như xác thực người dùng, tính toán khoảng cách địa lý, tích hợp trí tuệ nhân tạo, và quản lý cơ sở dữ liệu. Đồ án đã lựa chọn một bộ công nghệ phía máy chủ mạnh mẽ và linh hoạt để đáp ứng các yêu cầu này.

3.2.1 Node.js và Express.js

Node.js là nền tảng thực thi JavaScript phía máy chủ được xây dựng trên công cụ JavaScript V8 của Google Chrome. Node.js cho phép sử dụng JavaScript cho cả giao diện người dùng và phía máy chủ, giúp giảm thiểu sự phức tạp khi phải làm việc với nhiều ngôn ngữ lập trình khác nhau. Node.js đặc biệt phù hợp cho các ứng dụng tập trung vào nhập xuất như hệ thống này vì sử dụng mô hình hướng sự kiện, nhập xuất không chặn, giúp tăng thông lượng và khả năng xử lý đồng thời nhiều yêu cầu.

Express.js là khung công tác web nhẹ và linh hoạt được xây dựng trên Node.js, giúp xây dựng các giao diện lập trình ứng dụng RESTful một cách dễ dàng và có cấu trúc. Express.js cung cấp hệ thống phần mềm trung gian mạnh mẽ, cho phép xử lý các mối quan tâm xuyên suốt như xác thực, ghi nhật ký, và xử lý lỗi một cách tập trung. Trong đồ án, Express.js được sử dụng để định nghĩa các tuyến đường cho các điểm cuối giao diện lập trình ứng dụng như `/api/auth`, `/api/restaurants`, `/api/chatbot`, v.v.

3.2.2 PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ được chọn làm cơ sở dữ liệu chính của hệ thống. PostgreSQL được lựa chọn vì khả năng lưu trữ dữ liệu có cấu trúc ổn định cho hàng nghìn quán ăn và hàng chục nghìn đánh giá, đồng thời đảm bảo tính toàn vẹn dữ liệu thông qua tuân thủ ACID. PostgreSQL cũng hỗ trợ kiểu dữ liệu JSONB, cho phép lưu trữ dữ liệu bán cấu trúc như cấu hình

giờ mở cửa hoặc ngữ cảnh chatbot một cách hiệu quả. Trong đồ án, PostgreSQL lưu trữ 8 bảng chính bao gồm users, user_profiles, restaurants, reviews, favorites, blocks, chatbot_intents, và chatbot_conversations. Mỗi bảng được thiết kế với các ràng buộc phù hợp để đảm bảo tính toàn vẹn dữ liệu, và PostgreSQL hỗ trợ các chỉ mục để tối ưu hiệu năng truy vấn.

3.2.3 Sequelize ORM

Sequelize là một thư viện ánh xạ quan hệ đối tượng cho Node.js, giúp tương tác với PostgreSQL thông qua các đối tượng JavaScript thay vì phải viết SQL trực tiếp. Trong đồ án, Sequelize đóng vai trò là lớp trừu tượng giữa mã ứng dụng và cơ sở dữ liệu, giúp mã nguồn rõ ràng và dễ bảo trì hơn. Sequelize cung cấp cơ chế di chuyển để quản lý thay đổi lược đồ cơ sở dữ liệu một cách có kiểm soát, và cơ chế liên kết để định nghĩa mối quan hệ giữa các mô hình một cách rõ ràng. Sequelize cũng hỗ trợ xác thực ở cấp mô hình, đảm bảo dữ liệu được xác thực trước khi lưu vào cơ sở dữ liệu.

3.2.4 JWT và bcryptjs

JWT và bcryptjs là hai thư viện quan trọng đảm bảo an toàn cho hệ thống thông qua cơ chế xác thực dựa trên mã thông báo và mã hóa mật khẩu. JWT được sử dụng để tạo và xác minh mã thông báo xác thực người dùng. Khi người dùng đăng nhập thành công, phía máy chủ tạo một mã thông báo JWT chứa thông tin user_id và role, sau đó trả về cho giao diện người dùng. Phía máy chủ sử dụng phần mềm trung gian để xác minh mã thông báo và trích xuất thông tin người dùng, không cần phải truy vấn cơ sở dữ liệu mỗi lần, giúp giảm tải cho cơ sở dữ liệu và cải thiện hiệu năng. JWT có ưu điểm là không trạng thái, giúp hệ thống dễ mở rộng và có thể triển khai trên nhiều máy chủ mà không cần chia sẻ bộ lưu trữ phiên.

bcryptjs được sử dụng để băm mật khẩu người dùng trước khi lưu vào cơ sở dữ liệu. bcryptjs sử dụng thuật toán bcrypt với số vòng muối, đảm bảo mỗi mật khẩu được băm với một muối ngẫu nhiên khác nhau, giúp bảo vệ mật khẩu khỏi các cuộc tấn công bảng cầu vồng. Trong đồ án, bcryptjs được cấu hình với 10 vòng muối, đảm bảo cân bằng giữa bảo mật và hiệu năng.

3.3 Trí tuệ nhân tạo và Xử lý ngôn ngữ tự nhiên

Đây là thành phần then chốt tạo nên sự đột phá của đồ án so với các ứng dụng bản đồ thông thường. Khả năng hiểu và phản hồi ngôn ngữ tự nhiên của chatbot giúp người dùng tìm kiếm quán ăn một cách tự nhiên và trực quan hơn so với việc sử dụng biểu mẫu tìm kiếm truyền thống.

3.3.1 Google Gemini API

Google Gemini API được chọn làm nhà cung cấp trí tuệ nhân tạo chính để cung cấp khả năng tư vấn ẩm thực thông minh cho chatbot. Gemini là mô hình trí tuệ nhân tạo đa phương thức mới nhất của Google, được thiết kế để hiểu và xử lý ngôn ngữ tự nhiên một cách tốt hơn so với các mô hình trước đó. Trong đề án, Gemini được sử dụng với mô hình gemini-2.5-flash-lite, một mô hình được tối ưu hóa cho tốc độ và hiệu quả, phù hợp với ứng dụng thời gian thực như chatbot.

Lựa chọn Gemini API có nhiều ưu điểm. Đầu tiên, Gemini cung cấp hạn mức miễn phí khá lớn, phù hợp với dự án học thuật và phát triển. Thứ hai, Gemini có khả năng hiểu ngữ cảnh tiếng Việt tốt, đặc biệt quan trọng khi chatbot cần xử lý các câu hỏi bằng tiếng Việt của người dùng. Thứ ba, Gemini hỗ trợ lời nhắc hệ thống, cho phép định nghĩa vai trò và hành vi của chatbot một cách rõ ràng, giúp chatbot phản hồi phù hợp với mục đích của ứng dụng. Trong đề án, Gemini API được gọi thông qua Dịch vụ Chatbot Trí tuệ nhân tạo, nhận lời nhắc hệ thống chứa thông tin về các quán ăn phù hợp và ngữ cảnh người dùng, sau đó trả về phản hồi tự nhiên và danh sách quán ăn được gợi ý.

3.3.2 OpenAI API

OpenAI API được triển khai như một cơ chế dự phòng để đảm bảo chatbot luôn hoạt động ngay cả khi dịch vụ chính gặp sự cố hoặc vượt quá giới hạn truy cập. Hệ thống được cấu hình để tự động chuyển sang OpenAI API khi Gemini API không khả dụng hoặc trả về lỗi. OpenAI API sử dụng mô hình gpt-3.5-turbo, một mô hình mạnh mẽ và đã được kiểm chứng trong nhiều ứng dụng thực tế. Hệ thống được thiết kế để có thể chuyển đổi giữa Gemini và OpenAI một cách tự động và trong suốt, người dùng không nhận thấy sự khác biệt.

3.3.3 OpenStreetMap API

OpenStreetMap API là nguồn dữ liệu bản đồ cộng đồng khổng lồ được khai thác qua các tập lệnh thu thập để lấy thông tin vị trí và thuộc tính của hàng nghìn quán ăn. OpenStreetMap là một dự án bản đồ mã nguồn mở được xây dựng bởi cộng đồng, cung cấp dữ liệu địa lý miễn phí và mở cho mọi người sử dụng. Trong đề án, OpenStreetMap được sử dụng làm nguồn dữ liệu chính để thu thập thông tin về các quán ăn tại Hà Nội. Các tập lệnh thu thập được phát triển để truy vấn OpenStreetMap Overpass API, tìm kiếm các địa điểm có nhãn "amenity=restaurant" hoặc "amenity=cafe" trong khu vực Hà Nội. Thông tin này sau đó được làm sạch, chuẩn hóa, và lưu vào cơ sở dữ liệu của hệ thống.

Chương này đã giới thiệu toàn bộ hệ sinh thái công nghệ được áp dụng trong đề án, từ ReactJS ở Frontend đến Node.js, PostgreSQL ở Backend và các giải pháp AI

tiên tiến từ Google/OpenAI. Việc lựa chọn các công nghệ mã nguồn mở kết hợp với các API mạnh mẽ giúp hệ thống đạt được sự cân bằng giữa hiệu năng, chi phí và tính sáng tạo. Dựa trên nền tảng công nghệ này, Chương 4 sẽ trình bày chi tiết về quá trình thiết kế kiến trúc và triển khai thực tế ứng dụng.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 3 đã giới thiệu các công nghệ và nền tảng lý thuyết được sử dụng trong đề án. Chương 4 này sẽ trình bày chi tiết về quá trình hiện thực hóa hệ thống, bao gồm thiết kế kiến trúc, thiết kế chi tiết các thành phần, quá trình xây dựng mã nguồn và cuối cùng là công tác kiểm thử, triển khai ứng dụng.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Đề án lựa chọn kiến trúc MVC (Model-View-Controller) kết hợp với kiến trúc ba lớp (3-Tier Architecture). Đây là một biến thể của MVC được gọi là **Layered MVC** hoặc **MVC với Services Layer**.

Kiến trúc MVC là một mô hình kiến trúc phần mềm chia ứng dụng thành ba thành phần chính là Model, View và Controller. Trước tiên, phần Model (Mô hình) đại diện cho dữ liệu và logic nghiệp vụ, quản lý trạng thái và quy tắc xử lý dữ liệu, tương tác với database. Tiếp theo, phần View (Giao diện) đảm nhận vai trò hiển thị dữ liệu cho người dùng, nhận input từ người dùng, và không chứa logic nghiệp vụ. Cuối cùng, phần Controller (Điều khiển) sẽ xử lý input từ người dùng, điều phối giữa Model và View, đồng thời xử lý logic điều hướng.

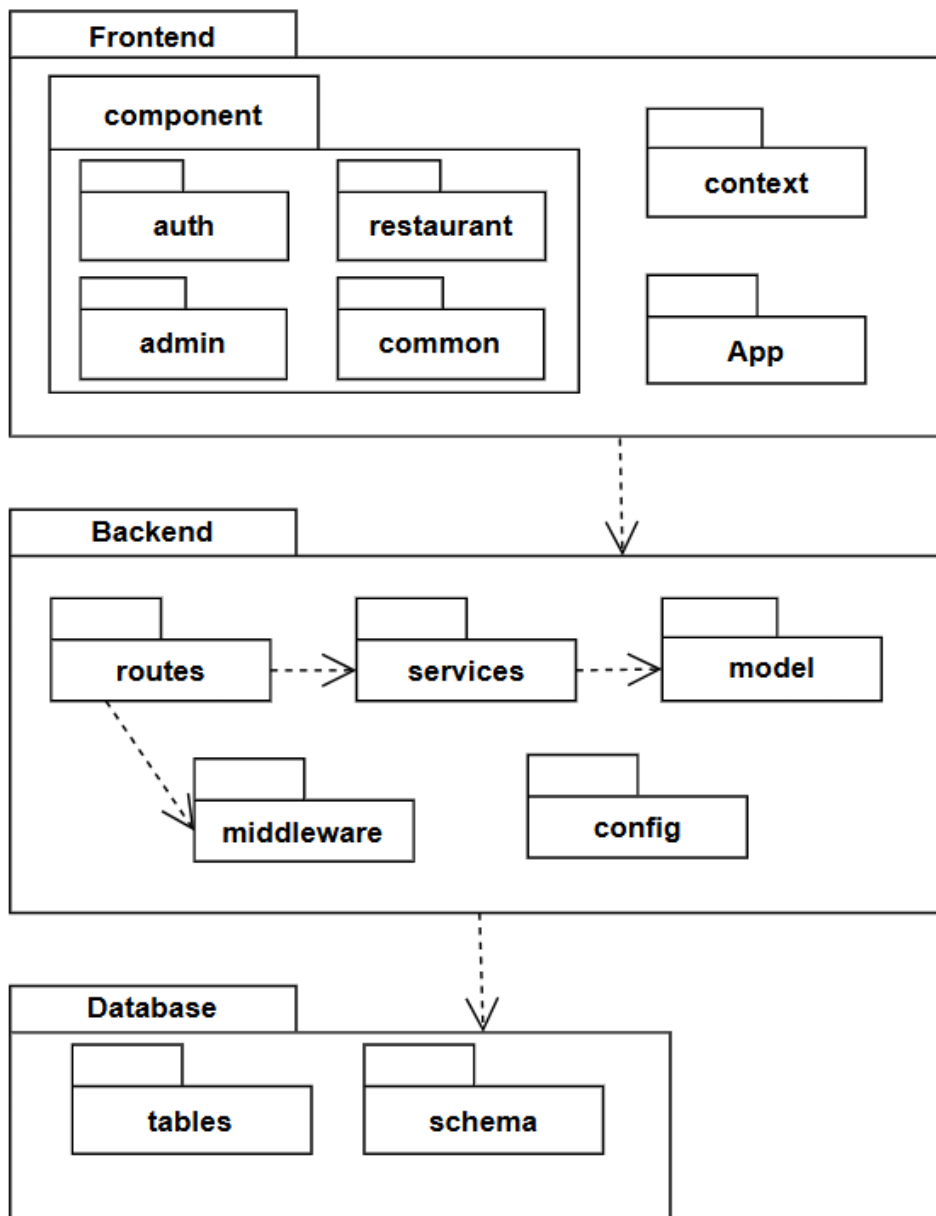
Hệ thống áp dụng kiến trúc MVC thông qua ba lớp chính. Thứ nhất, lớp Presentation (Frontend) tương đương với View trong MVC, được xây dựng bằng React.js với kiến trúc Component-based. Các thành phần UI như đăng nhập, chi tiết quán ăn, và Chatbot đóng vai trò là View, hiển thị dữ liệu và nhận đầu vào từ người dùng. Mỗi thành phần là một đơn vị độc lập với trạng thái riêng. Hệ thống sử dụng React Context API để quản lý trạng thái toàn cục như user, token, và is_authenticated. React Router DOM được sử dụng để quản lý navigation và URL state. Thứ hai, lớp Business Logic (Backend) tương đương với Controller và Services trong MVC. Lớp Routes (Controller) nhận các yêu cầu HTTP, xác nhận đầu vào, gọi Services hoặc Model, và trả về các phản hồi HTTP. Lớp Services (bổ sung) chứa logic nghiệp vụ phức tạp được tách ra khỏi Controller để dễ bảo trì và tái sử dụng. Lớp Middleware xử lý xác thực, phân quyền và chặn yêu cầu trước khi đến Routes. Thứ ba, lớp Data (Database) tương đương với Model persistence trong MVC. Các Sequelize Models định nghĩa cấu trúc dữ liệu, xác nhận dữ liệu, và tương tác với database thông qua Sequelize ORM; PostgreSQL Database lưu trữ dữ liệu.

Đề án có một số cải tiến so với MVC thuần túy. Đầu tiên, đề án bổ sung lớp Services để tách logic nghiệp vụ phức tạp như AI chatbot ra khỏi Controller, giúp

Controller gọn nhẹ hơn, logic có thể tái sử dụng, và dễ kiểm thử. Thứ hai, đồ án kết hợp với 3-Tier Architecture, cho phép Frontend và Backend chạy độc lập và giao tiếp qua RESTful API, từ đó có thể mở rộng từng phần độc lập. Thứ ba, đồ án sử dụng React Context API và local state để quản lý trạng thái phía client, giảm số lượng lời gọi API và cải thiện UX. Cuối cùng, đồ án tích hợp với các dịch vụ AI ngoài như Google Gemini và OpenAI thông qua lớp Services.

4.1.2 Thiết kế tổng quan

Biểu đồ gói UML của hệ thống được trình bày trong Hình 4.1. Hệ thống được chia thành ba gói chính: gói Frontend chứa các React components, context, và routing logic; gói Backend chứa routes, services, models, middleware, và config; và gói Database chứa schema và tables của PostgreSQL.



Hình 4.1: Biểu đồ phụ thuộc gói UML

Trong gói Frontend có các gói con gồm component, context và App. Gói component chứa các gói con bao gồm: gói auth chứa các component xác thực như Login, Register, ProfilePage, và ProfileSetup; gói restaurant chứa các component quán ăn như RestaurantListPage, RestaurantDetailPage, RecommendationsPage, và FavoritesPage; gói admin chứa các component quản trị như AdminPanel, User-ManagementPage, ReviewManagementPage, AddRestaurant, và EditRestaurant; gói common chứa các component dùng chung như Header, MapView, Chatbot, và LoadingScreen. Gói context quản lý state toàn cục thông qua AuthContext, còn gói App là component chính với routing.

Gói Backend chứa các gói con gồm: gói routes chứa các API endpoints như auth,

restaurants, reviews, chatbot, favorites, blocks, profile, recommendations, admin, map, và learning; gói services chứa business logic như `ai_chatbot_service`, `chatbot_service`, `recommendation_service`, và `restaurant_learning_service`; gói models chứa các Sequelize models như `User`, `UserProfile`, `Restaurant`, `Review`, `Favorite`, `Block`, `ChatbotIntent`, và `ChatbotConversation`; gói middleware xử lý cross-cutting concerns như `auth middleware`; và gói config chứa các file cấu hình như `database` và `associations`.

Trong gói Database, có hai gói chính schema và tables của PostgreSQL. Các thành phần chính bao gồm 8 bảng dữ liệu chính là `users` lưu thông tin tài khoản người dùng, `user_profiles` lưu hồ sơ và sở thích của người dùng, `restaurants` lưu thông tin các quán ăn, `reviews` lưu các đánh giá của người dùng về quán ăn, `favorites` lưu danh sách quán ăn yêu thích, `blocks` lưu danh sách quán ăn bị chặn, `chatbot_intents` lưu các ý định của chatbot, và `chatbot_conversations` lưu lịch sử trò chuyện. Ngoài ra, gói này còn chứa các indexes được tạo để tối ưu hiệu năng truy vấn, các khóa ngoài để đảm bảo tính toàn vẹn dữ liệu, và file `schema.sql` chứa định nghĩa toàn bộ cấu trúc database.

Về quy tắc phụ thuộc, Frontend phụ thuộc Backend thông qua việc gọi API qua HTTP/REST, trong khi Backend phụ thuộc Database thông qua query qua Sequelize ORM. Routes phụ thuộc Services và Models, Services phụ thuộc Models, và Routes sử dụng Middleware cho authentication. Quan trọng là không có phụ thuộc ngược, tức là Database không phụ thuộc Backend và Backend không phụ thuộc Frontend. Đồng thời, không có phụ thuộc bỏ qua tầng, nghĩa là Frontend không trực tiếp truy cập Database.

4.1.3 Thiết kế chi tiết gói

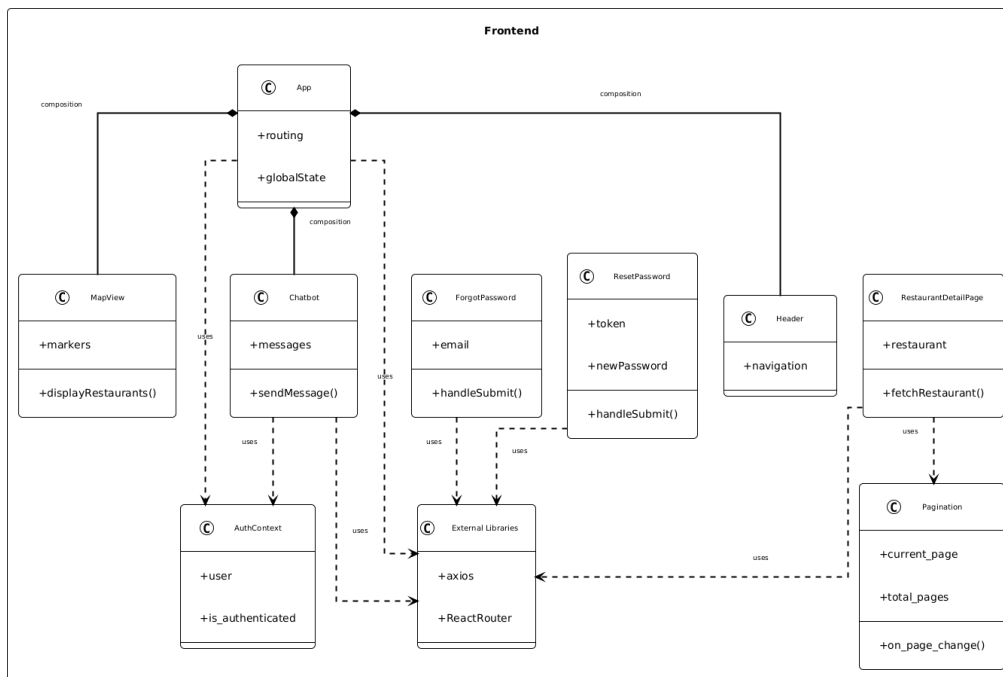
a, Thiết kế gói Frontend

Biểu đồ thiết kế gói Frontend được trình bày trong Hình 4.2. Gói Frontend sử dụng kiến trúc dựa trên thành phần với React.js. Các lớp chính bao gồm `App` là thành phần chính quản lý điều hướng và trạng thái toàn cục, `Header` là thành phần đầu trang hiển thị điều hướng và thông tin người dùng, `MapView` là thành phần bản đồ sử dụng React-Leaflet, `Chatbot` là thành phần chatbot xử lý tương tác với người dùng, `RestaurantDetailPage` là trang chi tiết quán ăn, `ForgotPassword` là trang quên mật khẩu cho phép người dùng nhập email để yêu cầu đặt lại mật khẩu, `ResetPassword` là trang đặt lại mật khẩu cho phép người dùng nhập mật khẩu mới với token hợp lệ, `Pagination` là thành phần tái sử dụng để hiển thị phân trang trên các trang danh sách, `AuthContext` là ngữ cảnh quản lý trạng thái xác thực, và `External Libraries` là nhóm các thư viện bên ngoài như `axios` và `ReactRouter`.

Về quan hệ giữa các lớp, App kết hợp với Header, MapView, và Chatbot thông qua quan hệ kết hợp. Điều này có nghĩa là App chứa các thành phần này như các phần tử con, và khi App bị hủy thì các thành phần con cũng bị hủy. App phụ thuộc vào AuthContext để quản lý trạng thái xác thực toàn cục, cho phép các thành phần ở nhiều cấp độ khác nhau có thể truy cập thông tin người dùng mà không cần truyền thuộc tính qua nhiều tầng. App cũng phụ thuộc vào External Libraries để sử dụng các thư viện như ReactRouter cho điều hướng và axios cho các yêu cầu HTTP.

RestaurantDetailPage phụ thuộc vào External Libraries để gọi API lấy thông tin quán ăn và đánh giá. Chatbot phụ thuộc vào AuthContext để lấy thông tin người dùng cho phiên trò chuyện, và phụ thuộc vào External Libraries để gửi tin nhắn đến Backend. ForgotPassword phụ thuộc vào External Libraries để gửi yêu cầu đặt lại mật khẩu đến Backend thông qua API POST /api/auth/forgot-password. ResetPassword phụ thuộc vào External Libraries để gửi mật khẩu mới đến Backend thông qua API POST /api/auth/reset-password với token hợp lệ. Pagination là thành phần tái sử dụng được sử dụng bởi các trang danh sách như RestaurantListPage, FavoritesPage, và RecommendationsPage để hiển thị điều khiển phân trang, cho phép người dùng chuyển trang và xem số trang hiện tại. Tất cả các thành phần phụ thuộc vào External Libraries thông qua axios để thực hiện các yêu cầu HTTP tới Backend, và thông qua ReactRouter để điều hướng giữa các trang.

Kiến trúc này cho phép tách biệt rõ ràng giữa các thành phần giao diện, logic điều hướng, quản lý trạng thái, và tương tác với Backend. Mỗi thành phần có trách nhiệm riêng và có thể được phát triển, kiểm thử, và bảo trì độc lập. Việc sử dụng React Context API giúp quản lý trạng thái toàn cục một cách hiệu quả mà không cần truyền props qua nhiều tầng, giúp mã nguồn gọn gàng và dễ bảo trì hơn.



Hình 4.2: Thiết kế gói Frontend

b, Thiết kế gói Backend

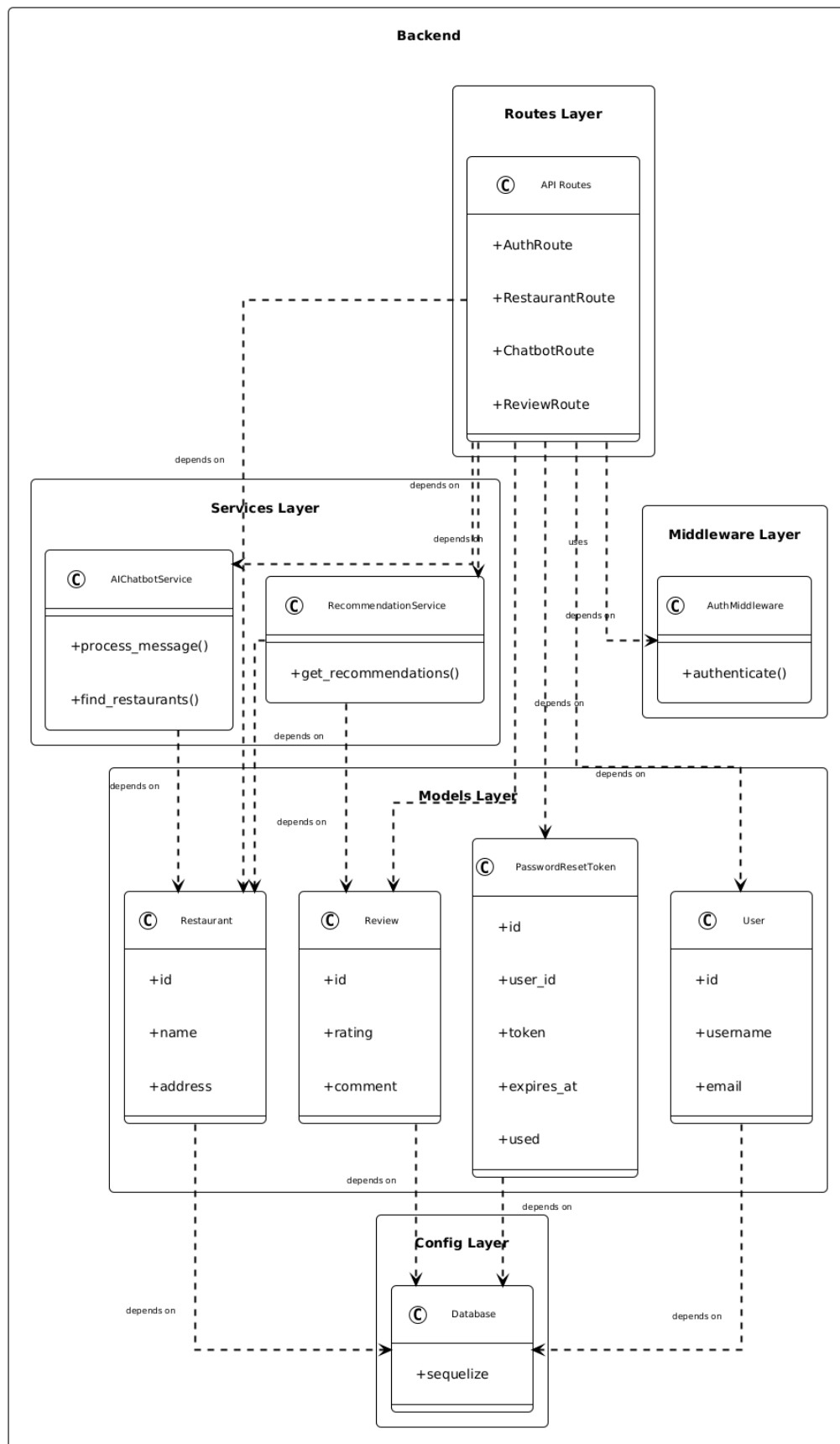
Biểu đồ thiết kế gói Backend được trình bày trong Hình 4.3. Gói Backend sử dụng kiến trúc phân lớp. Các lớp chính được tổ chức thành năm tầng: Tầng Routes, Tầng Services, Tầng Models, Tầng Middleware, và Tầng Config.

Tầng Routes chứa các trình xử lý route như API Routes, bao gồm AuthRoute, RestaurantRoute, ChatbotRoute, và ReviewRoute. AuthRoute xử lý các yêu cầu xác thực bao gồm đăng ký, đăng nhập, quên mật khẩu, đặt lại mật khẩu, và lấy thông tin người dùng hiện tại. Các route này đóng vai trò là Controller, nhận yêu cầu HTTP từ Frontend, xác thực đầu vào, gọi Service hoặc Model, và trả về phản hồi HTTP. Tầng Services chứa các lớp service như AIChatbotService và RecommendationService. AIChatbotService xử lý logic chatbot và tích hợp với Google Gemini và OpenAI API, trong khi RecommendationService xử lý logic gợi ý quán ăn dựa trên hồ sơ người dùng và đánh giá. Tầng Models chứa các Model Sequelize như User, Restaurant, Review, và PasswordResetToken, định nghĩa cấu trúc dữ liệu và tương tác với Database. PasswordResetToken lưu trữ các token để đặt lại mật khẩu với thời gian hết hạn và trạng thái sử dụng. Tầng Middleware chứa AuthMiddleware xử lý xác thực và phân quyền. Tầng Config chứa Database là kết nối Database thông qua Sequelize.

Về quan hệ, Routes phụ thuộc vào Services và Models. Ví dụ, ChatbotRoute phụ thuộc vào AIChatbotService để xử lý tin nhắn từ người dùng, và ReviewRoute phụ thuộc vào RecommendationService để lấy các gợi ý quán ăn. Routes cũng phụ

thuộc vào Models để truy cập dữ liệu, ví dụ AuthRoute phụ thuộc vào User và PasswordResetToken để xử lý quên mật khẩu và đặt lại mật khẩu, RestaurantRoute phụ thuộc vào Restaurant, và ReviewRoute phụ thuộc vào Review và Restaurant. Services phụ thuộc vào Models để truy cập dữ liệu, ví dụ AIChatbotService phụ thuộc vào Restaurant để tìm quán ăn phù hợp, và RecommendationService phụ thuộc vào Restaurant và Review để tính toán gợi ý. Routes sử dụng Middleware cho xác thực, tất cả các route đều sử dụng AuthMiddleware để xác thực người dùng trước khi xử lý yêu cầu, ngoại trừ các route công khai như đăng ký, đăng nhập, quên mật khẩu, và đặt lại mật khẩu. Models phụ thuộc vào Database, tất cả các Model như User, Restaurant, Review, và PasswordResetToken đều phụ thuộc vào Database để lưu trữ và truy xuất dữ liệu.

Kiến trúc phân lớp này đảm bảo tách biệt rõ ràng giữa các tầng, mỗi tầng có trách nhiệm riêng và chỉ phụ thuộc vào các tầng bên dưới. Điều này giúp mã nguồn dễ đọc, dễ bảo trì, và dễ mở rộng. Việc tách logic nghiệp vụ phức tạp vào Tầng Dịch vụ giúp Controller gọn nhẹ hơn và logic có thể tái sử dụng. Việc sử dụng Phần mềm trung gian cho xác thực giúp xử lý các mối quan tâm xuyên suốt một cách tập trung và nhất quán.



Hình 4.3: Thiết kế gói Backend

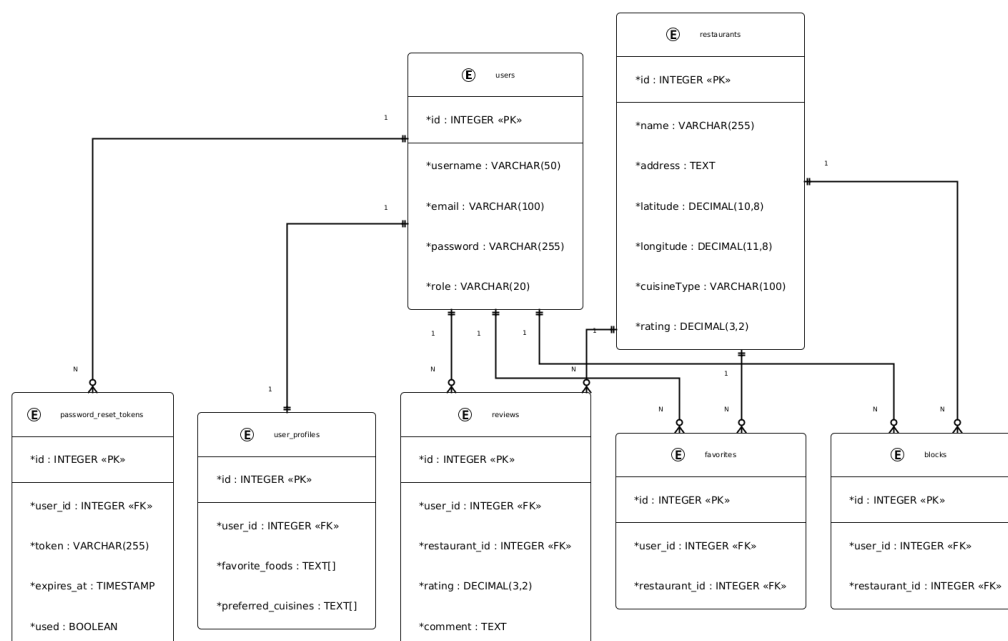
c, Thiết kế gói Database

Biểu đồ thiết kế gói Database được trình bày trong Hình 4.4. Gói Database sử dụng kiến trúc cơ sở dữ liệu quan hệ với PostgreSQL. Các thành phần chính bao gồm 7 bảng dữ liệu được tổ chức theo mô hình quan hệ. Bảng users lưu trữ thông tin tài khoản người dùng với các trường như id, username, email, password, và role. Bảng user_profiles lưu trữ hồ sơ chi tiết của người dùng với các trường như user_id, favorite_foods, và preferred_cuisines. Bảng restaurants lưu trữ thông tin các quán ăn với các trường như id, name, address, latitude, longitude, cuisineType, và rating. Bảng reviews lưu trữ các đánh giá của người dùng về quán ăn với các trường như id, user_id, restaurant_id, rating, và comment. Bảng favorites lưu trữ danh sách quán ăn yêu thích của người dùng với các trường như id, user_id, và restaurant_id. Bảng blocks lưu trữ danh sách quán ăn bị chặn của người dùng với các trường như id, user_id, và restaurant_id. Bảng password_reset_tokens lưu trữ các token để đặt lại mật khẩu với các trường như id, user_id, token, expires_at, và used để đảm bảo token chỉ được sử dụng một lần và có thời gian hết hạn.

Về quan hệ giữa các bảng, bảng users có quan hệ 1:1 với user_profiles thông qua khóa ngoại user_id. Điều này có nghĩa là mỗi người dùng có một hồ sơ duy nhất, và mỗi hồ sơ thuộc về một người dùng duy nhất. Bảng users có quan hệ 1:N với reviews, favorites, blocks, và password_reset_tokens thông qua khóa ngoại user_id. Điều này có nghĩa là mỗi người dùng có thể viết nhiều đánh giá, yêu thích nhiều quán, chặn nhiều quán, và có nhiều token đặt lại mật khẩu (mỗi token cho một lần yêu cầu đặt lại mật khẩu). Bảng restaurants có quan hệ 1:N với reviews, favorites, và blocks thông qua khóa ngoại restaurant_id. Điều này có nghĩa là mỗi quán có thể có nhiều đánh giá, được nhiều người dùng yêu thích, và bị nhiều người dùng chặn.

Các chỉ mục được tạo trên các trường thường được truy vấn như sessionId, createdAt, restaurant_id, user_id, latitude, longitude, cuisineType, và rating để tối ưu hiệu năng truy vấn. Các khóa ngoại đảm bảo tính toàn vẹn dữ liệu với ON DELETE CASCADE để tự động xóa các bản ghi liên quan khi bản ghi cha bị xóa. Ví dụ, khi một người dùng bị xóa, tất cả các đánh giá, yêu thích, và chặn của người dùng đó cũng sẽ tự động bị xóa.

Kiến trúc cơ sở dữ liệu quan hệ này đảm bảo tính nhất quán và toàn vẹn dữ liệu thông qua các ràng buộc và khóa ngoại. Việc sử dụng các chỉ mục giúp tối ưu hiệu năng truy vấn, đặc biệt là trên các trường thường được sử dụng để tìm kiếm và lọc dữ liệu. Các quan hệ được định nghĩa rõ ràng giúp đảm bảo tính toàn vẹn tham chiếu và tránh dữ liệu thừa thãi.



Hình 4.4: Thiết kế gói Database

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Hệ thống được thiết kế để hỗ trợ các độ phân giải màn hình khác nhau. Đối với máy tính để bàn, hệ thống được tối ưu cho độ phân giải 1920x1080 trở lên. Đối với máy tính bảng, hệ thống hỗ trợ độ phân giải từ 768px đến 1024px. Đối với thiết bị di động, hệ thống hỗ trợ cơ bản cho độ phân giải từ 320px đến 767px. Hệ thống hỗ trợ đầy đủ màu sắc 24-bit và sử dụng CSS3 với Flexbox và Grid layout để đảm bảo thiết kế phản hồi.

Đồ án đã thống nhất các chuẩn hóa khi thiết kế giao diện. Về thiết kế nút, nút chính sử dụng màu xanh dương với padding 12px 24px, border-radius 8px, và font-weight 600. Nút phụ sử dụng màu xám với cùng padding và border-radius. Nút nguy hiểm sử dụng màu đỏ với cùng kiểu. Tất cả các nút đều có hiệu ứng di chuột với opacity 0.8 và con trỏ con trỏ.

Về điều khiển, các trường đầu vào có viền 1px solid, border-radius 8px, padding 12px, và font-size 16px. Các menu thả xuống sử dụng kiểu tương tự các trường đầu vào. Hộp kiểm và nút radio có kích thước 20x20px với màu nhấn xanh dương.

Về vị trí hiển thị thông điệp phản hồi, thông báo thành công sử dụng màu xanh lá và hiển thị ở góc trên bên phải với hiệu ứng trượt vào. Thông báo lỗi sử dụng màu đỏ và hiển thị ở góc trên bên phải với hiệu ứng trượt vào. Chỉ báo tải là spinner màu xanh dương hiển thị ở trung tâm của thành phần.

Về phối màu, hệ thống sử dụng màu chính là xanh dương, màu phụ là xám nhạt

và xám đậm, màu thành công là xanh lá, màu cảnh báo là vàng cam, và màu lỗi là đỏ.

Về kiểu chữ, hệ thống sử dụng phông chữ hệ thống. Tiêu đề 1 có kích thước 32px với font-weight 700. Tiêu đề 2 có kích thước 24px với font-weight 600. Văn bản thân có kích thước 16px với font-weight 400. Văn bản nhỏ có kích thước 14px với font-weight 400.

Trang chủ bao gồm bản đồ tương tác ở bên trái và chatbot ở bên phải. Đầu trang hiển thị điều hướng và thông tin người dùng với các liên kết đến danh sách quán, gợi ý, yêu thích, và hồ sơ. Bản đồ sử dụng React-Leaflet để hiển thị điểm đánh dấu cho các quán ăn được gợi ý, cho phép người dùng tương tác bằng cách zoom, pan, và click vào các marker để xem thông tin chi tiết. Trang chi tiết quán ăn hiển thị thông tin đầy đủ về quán ăn bao gồm tên, địa chỉ, đánh giá, giờ mở cửa, mô tả, hình ảnh, và danh sách đánh giá. Trang này cũng có các nút "Yêu thích", "Chặn", và "Xem trên bản đồ" để người dùng tương tác, cùng với form để viết đánh giá mới. Trang đăng nhập và đăng ký sử dụng biểu mẫu đơn giản với đầu vào tên người dùng hoặc email và mật khẩu, có xác thực và hiển thị lỗi rõ ràng. Trang đăng nhập có liên kết đến trang quên mật khẩu để người dùng có thể khôi phục tài khoản. Trang quên mật khẩu cho phép người dùng nhập email để nhận link đặt lại mật khẩu, trong khi trang đặt lại mật khẩu cho phép người dùng nhập mật khẩu mới sau khi nhận được token hợp lệ.

Các trang danh sách quán ăn như Danh sách quán ăn, Yêu thích, và Gợi ý đều được tích hợp cơ chế phân trang. Mỗi trang hiển thị tối đa 12 quán ăn với component Pagination tái sử dụng, cho phép người dùng chuyển trang bằng nút "Trước"/"Sau" hoặc click trực tiếp vào số trang. Component Pagination tự động ẩn khi chỉ có 1 trang và hiển thị thông minh các số trang xung quanh trang hiện tại. Hệ thống tự động reset về trang 1 khi người dùng thay đổi bộ lọc hoặc loại gợi ý, đảm bảo trải nghiệm người dùng mượt mà.

Trang quản trị cung cấp bảng quản lý với các chức năng tạo, đọc, cập nhật, xóa cho quán ăn, người dùng, và đánh giá, kèm theo tính năng tìm kiếm và phân trang.

Bản Đồ Ẩm Thực	Quán ăn	Yêu thích	Gợi ý	Tài khoản	Đăng xuất
<Bản đồ (hiển thị vị trí người dùng)>				AI Chatbot	
				<Đoạn chat với người dùng>	
				<Gửi tin nhắn>	

Hình 4.5: Biểu đồ thiết kế giao diện trang chủ

Bản Đồ Ẩm Thực	Quán ăn	Yêu thích	Gợi ý	Tài khoản	Đăng xuất				
Danh sách Quán ăn <table border="1"> <tr> <td>Loại món</td> <td>Mức giá</td> <td>Đánh giá</td> <td> Tìm kiếm </td> </tr> </table> Quán ăn 1 Quán ăn 2 Quán ăn 3 ← Previous 1 2 3 ... 67 68 Next →						Loại món	Mức giá	Đánh giá	Tìm kiếm
Loại món	Mức giá	Đánh giá	Tìm kiếm						

Hình 4.6: Biểu đồ thiết kế giao diện Danh sách quán ăn

Bản Đồ Ẩm Thực	Quán ăn	Yêu thích	Gợi ý	Tài khoản	Đăng xuất
Thêm quán ăn mới					
<Form điền thông tin quán ăn>			<Bản đồ (hiển thị vị trí quán ăn)>		
Thêm quán ăn		Hủy			

Hình 4.7: Biểu đồ thiết kế giao diện Thêm quán ăn

4.2.2 Thiết kế lớp

Đồ án trình bày thiết kế chi tiết cho các lớp quan trọng nhất của hệ thống.

Lớp Dịch vụ Chatbot AI có mục đích xử lý logic chatbot trí tuệ nhân tạo và tích hợp với Google Gemini và OpenAI API. Lớp này chứa các thuộc tính như khóa giao diện lập trình ứng dụng OpenAI, khóa giao diện lập trình ứng dụng Gemini, nhà cung cấp đang hoạt động, mô hình Gemini mặc định, và mô hình OpenAI mặc định.

Lớp này cung cấp các phương thức quan trọng. Phương thức xử lý tin nhắn xử lý tin nhắn từ người dùng và trả về phản hồi trí tuệ nhân tạo cùng danh sách quán ăn gợi ý. Phương thức tìm quán ăn cho truy vấn tìm quán ăn phù hợp với truy vấn, lọc quán bị chặn, và tính khoảng cách. Phương thức tạo lời nhắc tạo lời nhắc hệ thống cho trí tuệ nhân tạo với thông tin quán ăn và ngữ cảnh người dùng. Phương thức gọi Gemini và gọi OpenAI gọi tương ứng Google Gemini API và OpenAI API. Phương thức loại bỏ trùng địa chỉ loại bỏ quán trùng địa chỉ và giữ quán mới nhất. Phương thức lấy danh sách ID quán bị chặn lấy danh sách ID quán bị chặn của người dùng.

Lớp Trang Chi tiết Quán ăn (Frontend) có mục đích hiển thị thông tin chi tiết về quán ăn và cho phép người dùng tương tác như yêu thích, chặn, và đánh giá. Lớp này quản lý các trạng thái như quán ăn chứa thông tin quán ăn, đánh giá chứa danh sách đánh giá, trạng thái yêu thích, trạng thái chặn, và trạng thái tải.

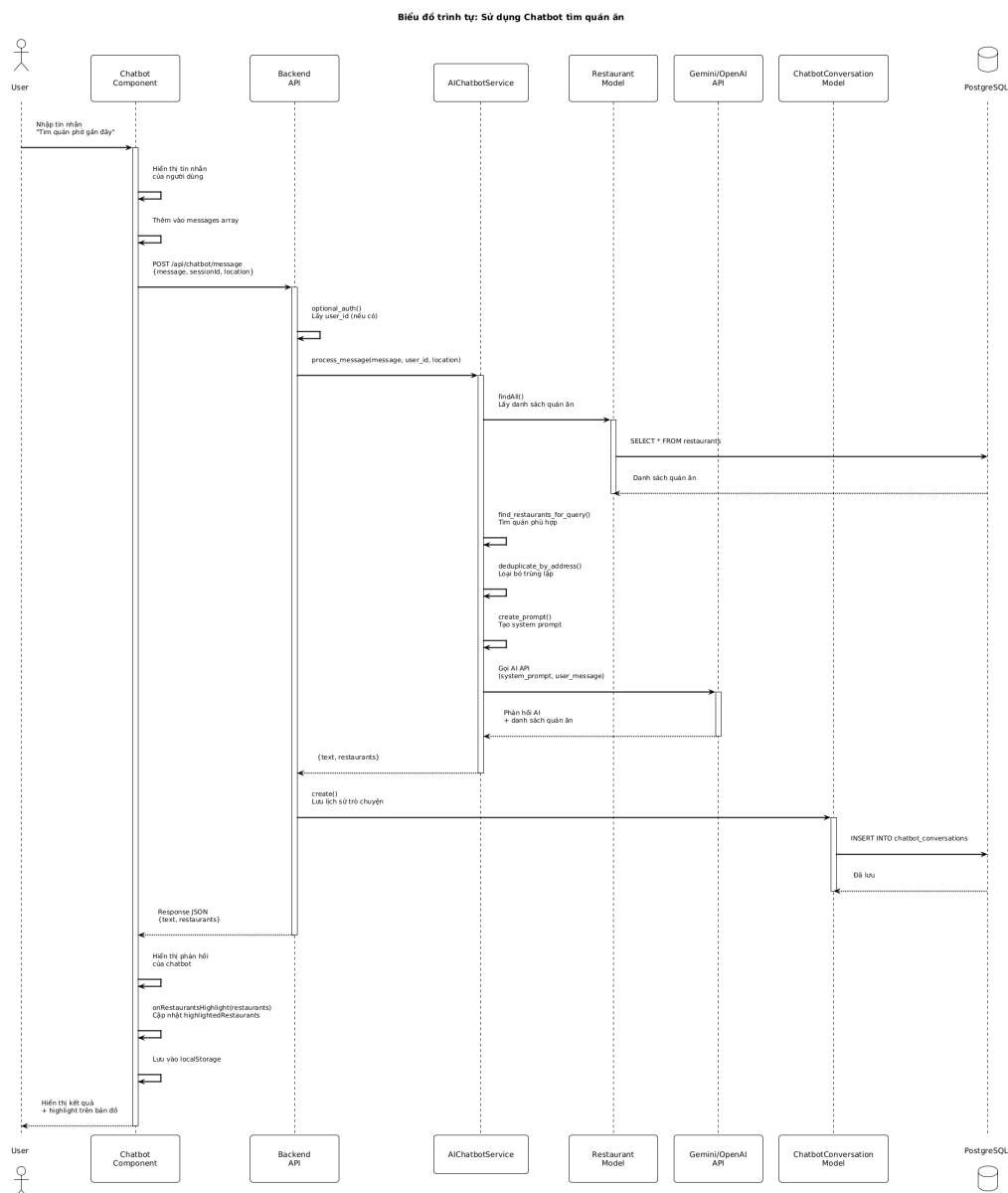
Lớp này cung cấp các phương thức như lấy quán ăn để lấy thông tin quán ăn từ giao diện lập trình ứng dụng, lấy đánh giá để lấy danh sách đánh giá từ giao

diện lập trình ứng dụng, kiểm tra yêu thích để kiểm tra quán có trong danh sách yêu thích không, kiểm tra chặn để kiểm tra quán có bị chặn không, chuyển đổi yêu thích để thêm hoặc xóa quán khỏi danh sách yêu thích, và chuyển đổi chặn để chặn hoặc bỏ chặn quán.

Lớp Đánh giá (Model) có mục đích định nghĩa cấu trúc dữ liệu và xác thực cho đánh giá quán ăn. Lớp này chứa các thuộc tính như id là khóa chính kiểu INTEGER với tự động tăng, user_id là khóa ngoại đến users kiểu INTEGER NOT NULL, restaurant_id là khóa ngoại đến restaurants kiểu INTEGER NOT NULL, rating là điểm đánh giá kiểu DECIMAL(3,2) trong khoảng 1-5 và NOT NULL, comment là nội dung đánh giá kiểu TEXT, images là mảng URL hình ảnh kiểu TEXT[], is_verified là đánh giá đã được xác minh kiểu BOOLEAN với mặc định false, và createdAt cùng updatedAt là dấu thời gian.

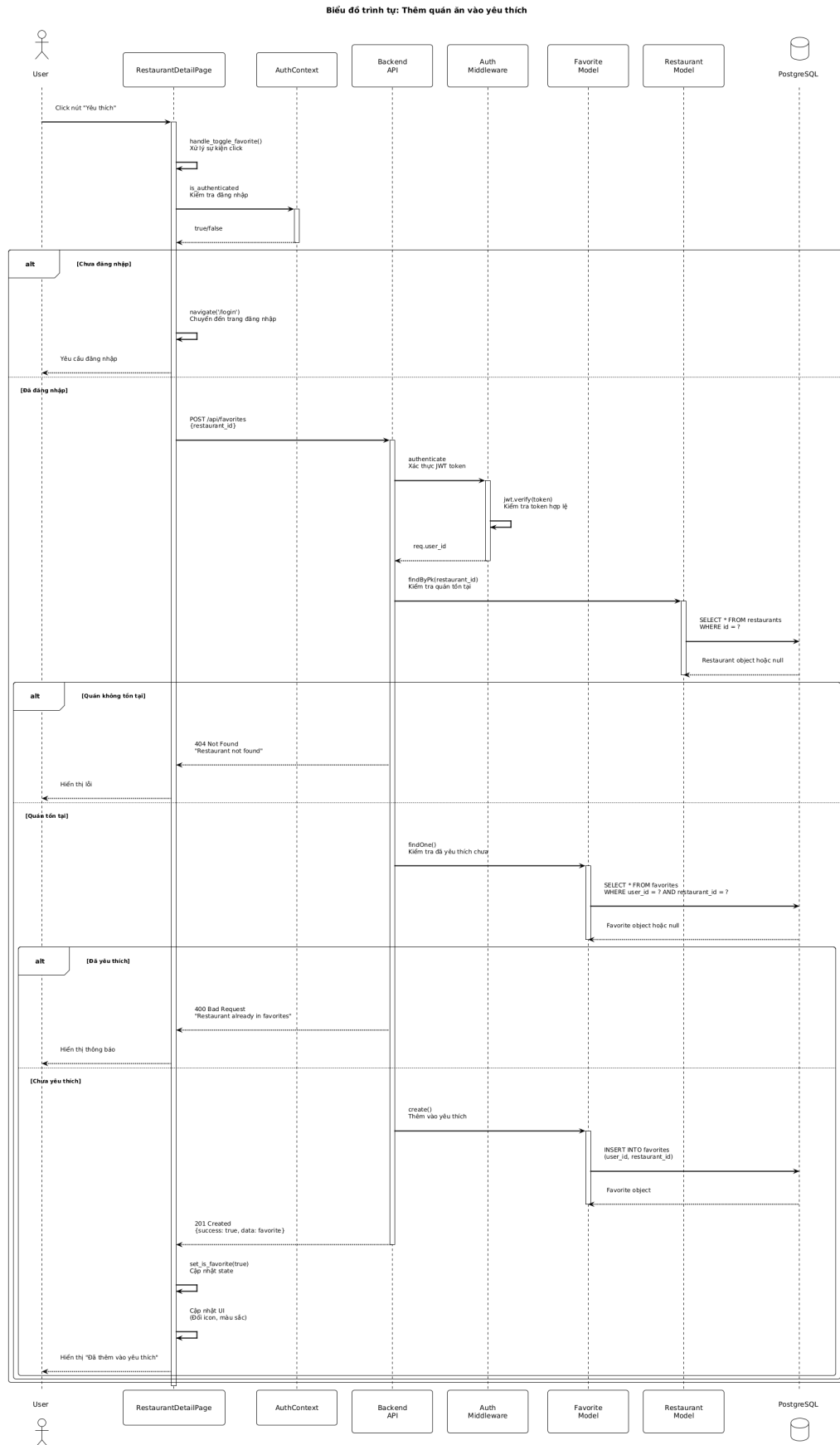
Về xác thực, rating phải trong khoảng 1-5, và user_id cùng restaurant_id phải tồn tại trong cơ sở dữ liệu.

Biểu đồ trình tự cho Use Case quan trọng "Sử dụng chatbot tìm quán ăn" được trình bày trong Hình 4.8.



Hình 4.8: Biểu đồ trình tự - Sử dụng chatbot tìm quán ăn

Biểu đồ trình tự cho Use Case "Thêm quán ăn yêu thích" được trình bày trong Hình 4.9.

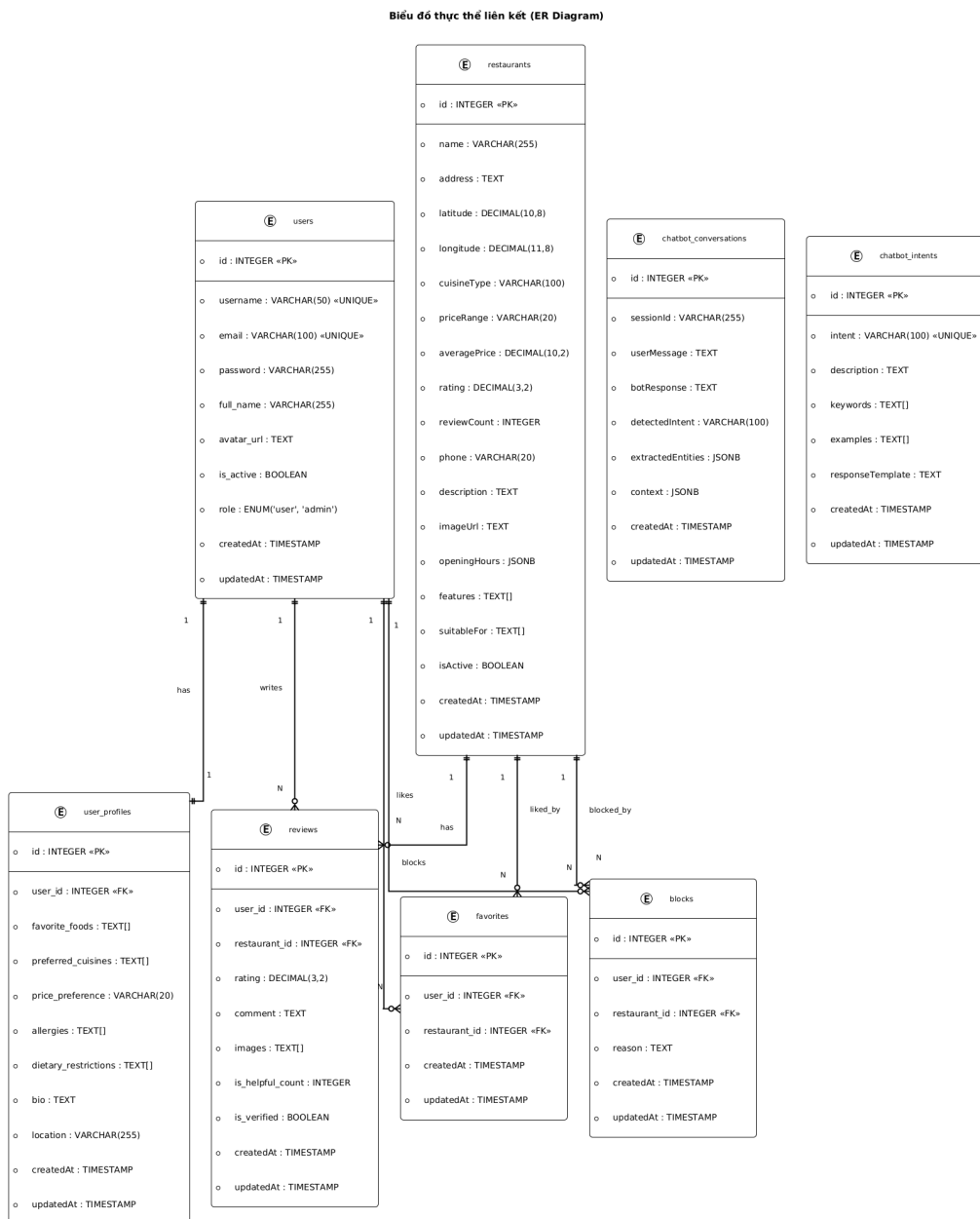


Hình 4.9: Biểu đồ trình tự - Thêm quán ăn yêu thích

4.2.3 Thiết kế cơ sở dữ liệu

Biểu đồ E-R của hệ thống được trình bày trong Hình 4.10. Hệ thống có 6 thực thể chính bao gồm User là người dùng hệ thống, UserProfile là hồ sơ người dùng chứa thông tin về món yêu thích, loại món ưa thích, mức giá, dị ứng, và hạn chế, Restaurant là quán ăn, Review là đánh giá quán ăn, Favorite là quán ăn yêu thích, và Block là quán ăn bị chặn.

Về quan hệ giữa các thực thể, User có quan hệ 1:1 với UserProfile, nghĩa là mỗi người dùng có một hồ sơ. User có quan hệ 1:N với Review, nghĩa là mỗi người dùng có thể viết nhiều đánh giá. Restaurant có quan hệ 1:N với Review, nghĩa là mỗi quán có thể có nhiều đánh giá. User có quan hệ 1:N với Favorite, nghĩa là mỗi người dùng có thể yêu thích nhiều quán. Restaurant có quan hệ 1:N với Favorite, nghĩa là mỗi quán có thể được nhiều người dùng yêu thích. User có quan hệ 1:N với Block, nghĩa là mỗi người dùng có thể chặn nhiều quán. Restaurant có quan hệ 1:N với Block, nghĩa là mỗi quán có thể bị nhiều người dùng chặn.



Hình 4.10: Biểu đồ thực thể liên kết (E-R Diagram)

Hệ thống sử dụng PostgreSQL 12+ làm hệ quản trị cơ sở dữ liệu. Bảng users chứa các trường như id là SERIAL PRIMARY KEY, username là VARCHAR(50) UNIQUE NOT NULL, email là VARCHAR(100) UNIQUE NOT NULL, password là VARCHAR(255) NOT NULL được băm với bcrypt, role là VARCHAR(20) DEFAULT 'user' với ràng buộc kiểm tra chỉ cho phép 'user' hoặc 'admin', và createdAt cùng updatedAt là TIMESTAMP.

Bảng restaurants chứa các trường như id là SERIAL PRIMARY KEY, name là VARCHAR(255) NOT NULL, address là TEXT NOT NULL, latitude là DECIMAL(10,8) NOT NULL, longitude là DECIMAL(11,8) NOT NULL, cuisineType là VARCHAR(100) NOT NULL, priceRange là VARCHAR(20) NOT NULL với

ràng buộc kiểm tra chỉ cho phép 'budget', 'moderate', 'expensive', hoặc 'luxury', rating là DECIMAL(3,2) DEFAULT 0 với ràng buộc kiểm tra đảm bảo rating từ 0 đến 5, reviewCount là INTEGER DEFAULT 0, openingHours là JSONB để lưu giờ mở cửa, features là TEXT[] chứa mảng các tính năng, suitableFor là TEXT[] chứa mảng các dịp phù hợp, và isActive là BOOLEAN DEFAULT true. Bảng này có các chỉ mục như idx_restaurants_location trên (latitude, longitude), idx_restaurants_cuisine trên cuisineType, và idx_restaurants_rating trên rating.

Bảng reviews chứa các trường như id là SERIAL PRIMARY KEY, user_id là INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE, restaurant_id là INTEGER NOT NULL REFERENCES restaurants(id) ON DELETE CASCADE, rating là DECIMAL(3,2) NOT NULL với ràng buộc kiểm tra đảm bảo rating từ 1 đến 5, comment là TEXT, images là TEXT[], is_verified là BOOLEAN DEFAULT false. Bảng này có các chỉ mục như idx_reviews_restaurant_id trên restaurant_id và idx_reviews_user_id trên user_id. Bảng này cũng có ràng buộc duy nhất trên (user_id, restaurant_id) để đảm bảo mỗi người dùng chỉ đánh giá một quán một lần.

Bảng favorites chứa các trường như id là SERIAL PRIMARY KEY, user_id là INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE, restaurant_id là INTEGER NOT NULL REFERENCES restaurants(id) ON DELETE CASCADE. Bảng này có ràng buộc duy nhất trên (user_id, restaurant_id) và các chỉ mục như idx_favorites_user_id trên user_id và idx_favorites_restaurant_id trên restaurant_id.

Bảng blocks chứa các trường như id là SERIAL PRIMARY KEY, user_id là INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE, restaurant_id là INTEGER NOT NULL REFERENCES restaurants(id) ON DELETE CASCADE, reason là VARCHAR(255). Bảng này có ràng buộc duy nhất trên (user_id, restaurant_id) và các chỉ mục như idx_blocks_user_id trên user_id và idx_blocks_restaurant_id trên restaurant_id.

Bảng chatbot_conversations chứa các trường như id là SERIAL PRIMARY KEY, sessionId là VARCHAR(255) NOT NULL, userMessage là TEXT NOT NULL, botResponse là TEXT NOT NULL, detectedIntent là VARCHAR(100), extractedEntities là JSONB DEFAULT '', và context là JSONB DEFAULT '' để lưu mảng quán ăn. Bảng này có các chỉ mục như idx_conversations_session trên sessionId, idx_conversations_created trên createdAt, và idx_conversations_session_created trên (sessionId, createdAt).

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.1 liệt kê các thư viện và công cụ chính được sử dụng trong đồ án.

Mục đích	Công cụ	Địa chỉ URL
IDE lập trình	Cursor	https://cursor.com/
Version Control	Git	https://git-scm.com/
Frontend Framework	React.js 18+	https://react.dev/
Build Tool	Vite 5.0+	https://vitejs.dev/
Backend Framework	Express.js 4.18+	https://expressjs.com/
Runtime	Node.js 16+	https://nodejs.org/
Database	PostgreSQL 12+	https://www.postgresql.org/
ORM	Sequelize 6.35+	https://sequelize.org/
HTTP Client	Axios 1.6+	https://axios-http.com/
Bản đồ	React-Leaflet 4.2+	https://react-leaflet.js.org/
AI API	Google Gemini API	https://ai.google.dev/
AI API (Fallback)	OpenAI API	https://openai.com/
NLP Library	Natural 6.10+	https://github.com/NaturalNode/natural
Authentication	JWT (jsonwebtoken)	https://github.com/auth0/node-jwt-token
Password Hashing	bcryptjs 2.4+	https://github.com/dcodeIO/bcrypt.js

Bảng 4.1: Danh sách thư viện và công cụ sử dụng

4.3.2 Kết quả đạt được

Đồ án đã xây dựng thành công một hệ thống web hoàn chỉnh với các thành phần chính. Phần Frontend được xây dựng bằng React.js với hơn 23 component được tổ chức theo mô đun như xác thực, quán ăn, quản trị, và dùng chung. Các component mới bao gồm ForgotPassword, ResetPassword, và Pagination. Hệ thống sử dụng React Router DOM v6 cho điều hướng, React Context API cho quản lý trạng thái, và React-Leaflet để tích hợp bản đồ. Giao diện được thiết kế phản hồi, hỗ trợ máy tính để bàn và máy tính bảng.

Phần Backend được xây dựng bằng Node.js với Express.js, cung cấp hơn 10 route API bao gồm xác thực, quán ăn, đánh giá, chatbot, yêu thích, chặn, hồ sơ, gợi ý, quản trị, bản đồ, và học. Các route xác thực bao gồm đăng ký, đăng nhập, quên mật khẩu, đặt lại mật khẩu, và lấy thông tin người dùng hiện tại. Các route danh sách như restaurants, favorites, và recommendations đều được cập nhật để hỗ trợ phân trang với tham số page và limit, trả về thông tin pagination gồm total, page, limit, total_pages, has_next, và has_prev. Hệ thống có 4 Service class là Chatbot Trí tuệ nhân tạo, Chatbot, Gợi ý, và Học. Hệ thống có 9 Model Sequelize bao gồm User, UserProfile, Restaurant, Review, Favorite, Block, ChatbotIntent, ChatbotConversation, và PasswordResetToken. Hệ thống sử dụng middleware xác thực với JWT và tích hợp Google Gemini và OpenAI API.

Phần Database sử dụng PostgreSQL với 9 bảng chính có đầy đủ quan hệ. Các bảng bao gồm users, user_profiles, restaurants, reviews, favorites, blocks,

chatbot_intents, chatbot_conversations, và password_reset_tokens. Bảng password_reset_tokens lưu trữ các token để đặt lại mật khẩu với thời gian hết hạn và trạng thái sử dụng. Hệ thống có chỉ mục để tối ưu hiệu năng, khóa ngoại để đảm bảo tính toàn vẹn dữ liệu, và schema được quản lý bằng Sequelize migrations.

Hệ thống cũng bao gồm hơn 15 tập lệnh tiện ích hỗ trợ như migration, seed, crawl dữ liệu, và kiểm tra dữ liệu. Các công cụ thu thập tự động thu thập dữ liệu từ OpenStreetMap, Wikidata, và Nominatim.

Bảng 4.2 thống kê các thông tin về mã nguồn của ứng dụng.

Thông số	Giá trị
Số dòng code (ước tính)	16,000+ dòng
Số component Frontend	23+ components
Số route Backend	10+ routes
Số Service class	4 services
Số Model	9 models
Số bảng Database	9 tables
Dung lượng mã nguồn Frontend	2 MB
Dung lượng mã nguồn Backend	3 MB
Dung lượng Database schema	50 KB
Tổng dung lượng mã nguồn	5 MB (không bao gồm node_modules)

Bảng 4.2: Thống kê mã nguồn

4.3.3 Minh họa các chức năng chính

Đồ án minh họa các chức năng chính của hệ thống. Trang chủ với Chatbot và Bản đồ hiển thị bản đồ tương tác ở bên trái và chatbot ở bên phải. Người dùng có thể tương tác với chatbot để tìm quán ăn, và các quán được gợi ý sẽ được làm nổi bật trên bản đồ với điểm đánh dấu màu đỏ. Chức năng tìm kiếm quán ăn qua Chatbot cho phép người dùng nhập câu hỏi như "Tìm quán phở gần đây", chatbot sử dụng trí tuệ nhân tạo (Gemini hoặc OpenAI) để phân tích và tìm quán phù hợp, sau đó hiển thị kết quả và làm nổi bật trên bản đồ. Chức năng xem chi tiết quán ăn hiển thị đầy đủ thông tin quán ăn, danh sách đánh giá, và các nút tương tác như Yêu thích, Chặn, và Xem trên bản đồ. Chức năng quản lý đánh giá cho phép người dùng xem và viết đánh giá cho quán ăn, trong khi quản trị viên có thể quản lý tất cả đánh giá, chỉnh sửa và xóa. Chức năng gợi ý cá nhân hóa gợi ý quán ăn dựa trên hồ sơ người dùng như món yêu thích, loại món ưa thích, mức giá, dị ứng, và hạn chế. Chức năng phân trang được tích hợp vào tất cả các trang danh sách, giúp cải thiện hiệu năng và trải nghiệm người dùng khi duyệt qua số lượng lớn quán ăn. Người dùng có thể dễ dàng chuyển trang và hệ thống tự động reset về trang 1 khi filter thay đổi.

4.4 Kiểm thử

Đồ án đã thực hiện kiểm thử cho các chức năng quan trọng nhất của hệ thống. Quá trình kiểm thử được thực hiện theo nhiều cấp độ khác nhau, từ kiểm thử đơn vị đến kiểm thử tích hợp và kiểm thử hệ thống, nhằm đảm bảo chất lượng và độ tin cậy của ứng dụng. Dưới đây trình bày 30 test case quan trọng nhất đã được thực hiện và xác nhận.

4.4.1 Chiến lược kiểm thử

Kiểm thử được thực hiện trên môi trường phát triển (Development). Đồ án áp dụng chiến lược kiểm thử đa tầng, bao gồm kiểm thử thủ công (Manual Testing) cho các chức năng chính. Do tính chất của đồ án là một hệ thống web tương tác với nhiều thành phần phức tạp như AI chatbot, bản đồ, và gợi ý cá nhân hóa, đồ án tập trung vào kiểm thử tích hợp và kiểm thử hệ thống để đảm bảo các thành phần hoạt động đúng khi kết hợp với nhau.

4.4.2 Kiểm thử chức năng xác thực

Kiểm thử chức năng xác thực đã được thực hiện cho các tính năng đăng ký, đăng nhập, quên mật khẩu, và bảo mật. Bảng 4.3 trình bày các test case chính đã được thực hiện cho chức năng xác thực.

Test Case ID	Mô tả	Dữ liệu đầu vào	Kết quả
TC-AUTH-001	Đăng ký tài khoản mới thành công	Username: "Linh", Email: "nhatlinhn-guyen3002@gmail.com", Password: "141203"	Đạt
TC-AUTH-002	Đăng ký với email đã tồn tại	Email: "nhatlinhn-guyen3002@gmail.com"	Đạt
TC-AUTH-003	Đăng nhập thành công	Username: "Linh", Password: "141203"	Đạt
TC-AUTH-004	Quên mật khẩu với email hợp lệ	Email: "nhatlinhn-guyen3002@gmail.com"	Đạt
TC-AUTH-005	Truy cập route bảo vệ không có token	Không có JWT token	Đạt

Bảng 4.3: Test cases cho chức năng xác thực

Kết quả kiểm thử cho thấy hệ thống xác thực cơ bản hoạt động đúng với kỳ vọng.

4.4.3 Kiểm thử chức năng quản lý quán ăn

Kiểm thử chức năng quản lý quán ăn đã được thực hiện cho các thao tác CRUD (Create, Read, Update, Delete) và các tính năng phân trang, phân quyền. Bảng 4.4 trình bày các test case chính đã được thực hiện.

Test Case ID	Mô tả	Dữ liệu đầu vào	Kết quả
TC-REST-001	Lấy danh sách quán ăn	GET /api/restaurants	Đạt
TC-REST-002	Lấy chi tiết quán ăn	GET /api/restaurants/:id	Đạt
TC-REST-003	Tạo quán ăn mới (Admin)	POST /api/restaurants với dữ liệu hợp lệ	Đạt

Bảng 4.4: Test cases cho chức năng quản lý quán ăn

Kết quả kiểm thử cho thấy hệ thống phân trang hoạt động chính xác và phân quyền được thực thi đúng để đảm bảo chỉ admin mới có thể tạo quán ăn.

4.4.4 Kiểm thử chức năng chatbot

Kiểm thử chức năng chatbot đã được thực hiện để kiểm tra tính phức tạp của việc tích hợp AI và xử lý ngôn ngữ tự nhiên. Bảng 4.5 trình bày các test case chính đã được thực hiện.

Test Case ID	Mô tả	Dữ liệu đầu vào	Kết quả
TC-CHAT-001	Chatbot tìm quán theo loại món	"Tìm quán phở"	Đạt
TC-CHAT-002	Chatbot tìm quán kết hợp	"Tìm quán phở gần đây"	Chưa đạt
TC-CHAT-003	Chatbot lọc quán bị chặn	User có quán bị chặn trong kết quả	Đạt
TC-CHAT-004	Chatbot khi API lỗi (fallback)	API key không hợp lệ	Đạt
TC-CHAT-005	Lưu lịch sử trò chuyện	Mỗi tin nhắn	Đạt

Bảng 4.5: Test cases cho chức năng chatbot

Kết quả kiểm thử cho thấy chatbot hoạt động tốt với khả năng hiểu các câu hỏi tiếng Việt và trả về kết quả phù hợp, tuy nhiên với câu hỏi phức tạp hơn chatbot vẫn gặp khó khăn trong việc xác định chính xác yêu cầu của người dùng. Hệ thống fallback giữa Gemini và OpenAI hoạt động đúng khi một API gặp lỗi. Chức năng lọc quán bị chặn và việc lưu lịch sử trò chuyện vào database hoạt động chính xác.

4.4.5 Kiểm thử chức năng đánh giá và yêu thích

Kiểm thử chức năng đánh giá và yêu thích đã được thực hiện để kiểm tra việc tạo đánh giá và quản lý danh sách yêu thích. Bảng 4.6 trình bày các test case chính đã được thực hiện.

Test Case ID	Mô tả	Dữ liệu đầu vào	Kết quả
TC-REV-001	Tạo đánh giá mới	Rating: 5, Comment: "Ngon"	Đạt
TC-REV-002	Thêm quán vào yêu thích	POST /api/favorites	Đạt
TC-REV-003	Xóa khỏi yêu thích	DELETE /api/favorites/:id	Đạt
TC-REV-004	Lấy danh sách yêu thích	GET /api/favorites	Đạt

Bảng 4.6: Test cases cho chức năng đánh giá và yêu thích

Kết quả kiểm thử cho thấy ràng buộc duy nhất hoạt động đúng, đảm bảo mỗi người dùng chỉ đánh giá một quán một lần và chỉ yêu thích một quán một lần.

4.4.6 Kiểm thử chức năng gợi ý cá nhân hóa

Kiểm thử chức năng gợi ý cá nhân hóa đã được thực hiện để kiểm tra hệ thống gợi ý quán ăn phù hợp dựa trên hồ sơ người dùng. Bảng 4.7 trình bày các test case chính đã được thực hiện.

Test Case ID	Mô tả	Dữ liệu đầu vào	Kết quả
TC-REC-001	Gợi ý loại trừ quán bị chặn	User có quán bị chặn	Đạt
TC-REC-002	Gợi ý kết hợp nhiều tiêu chí	User có đầy đủ profile	Đạt
TC-REC-003	Gợi ý cho user chưa có profile	User mới, chưa có profile	Đạt

Bảng 4.7: Test cases cho chức năng gợi ý cá nhân hóa

Kết quả kiểm thử cho thấy hệ thống gợi ý hoạt động tốt với khả năng kết hợp nhiều tiêu chí từ hồ sơ người dùng, tự động loại trừ quán bị chặn, và đưa ra gợi ý

phù hợp cho người dùng mới chưa có profile.

4.4.7 Kiểm thử chức năng quản trị

Kiểm thử chức năng quản trị đã được thực hiện để kiểm tra các tính năng quản lý và phân quyền. Bảng 4.8 trình bày các test case chính đã được thực hiện.

Test Case ID	Mô tả	Dữ liệu đầu vào	Kết quả
TC-ADM-001	Truy cập trang quản trị (Admin)	User có role: "admin"	Đạt
TC-ADM-002	Truy cập trang quản trị (User)	User có role: "user"	Đạt
TC-ADM-003	Quản lý quán ăn - Tìm kiếm	GET /api/admin/restaurants?search=phở	Đạt

Bảng 4.8: Test cases cho chức năng quản trị

Kết quả kiểm thử cho thấy phân quyền hoạt động đúng, đảm bảo chỉ người dùng có role "admin" mới có thể truy cập các trang và API quản trị.

4.4.8 Kiểm thử giao diện người dùng

Kiểm thử giao diện người dùng đã được thực hiện để kiểm tra tính phản hồi và trải nghiệm người dùng. Bảng 4.9 trình bày các test case chính đã được thực hiện.

Test Case ID	Mô tả	Thiết bị/Trình duyệt	Kết quả
TC-UI-001	Hiển thị trang chủ trên desktop	Chrome, Edge	Đạt
TC-UI-002	Bản đồ tương tác - Click marker	Click vào marker quán ăn	Đạt
TC-UI-003	Chatbot - Gửi tin nhắn	Nhập và gửi	Đạt

Bảng 4.9: Test cases cho giao diện người dùng

Kết quả kiểm thử cho thấy giao diện hoạt động tốt trên các trình duyệt khác nhau và các thành phần tương tác như bản đồ và chatbot hoạt động mượt mà.

4.4.9 Kiểm thử hiệu năng

Kiểm thử hiệu năng đã được thực hiện để kiểm tra khả năng hệ thống xử lý các yêu cầu trong thời gian hợp lý. Bảng 4.10 trình bày các test case chính đã được thực hiện.

Test Case ID	Mô tả	Thời gian phản hồi	Kết quả
TC-PERF-001	API lấy danh sách quán (12 items)	< 500ms	Đạt
TC-PERF-002	API chatbot (Gemini)	< 3s	Đạt

Bảng 4.10: Test cases cho hiệu năng

Kết quả kiểm thử cho thấy thời gian phản hồi của hệ thống nằm trong giới hạn chấp nhận được, đáp ứng yêu cầu về hiệu năng.

4.4.10 Tổng hợp kết quả kiểm thử

Bảng 4.11 tổng hợp số lượng test case đã được thực hiện cho các chức năng chính của hệ thống.

Chức năng	Số lượng test case
Xác thực	5
Quản lý quán ăn	3
Chatbot	5
Đánh giá và yêu thích	4
Gợi ý cá nhân hóa	3
Quản trị	2
Giao diện người dùng	3
Hiệu năng	2
Tổng cộng	27

Bảng 4.11: Tổng hợp số lượng test case đã thực hiện

Tổng cộng có 27 test case đã được thực hiện để kiểm tra các chức năng quan trọng nhất của hệ thống. Các test case này bao gồm các trường hợp kiểm thử validation, error handling, phân quyền, và các luồng nghiệp vụ chính. Kết quả kiểm thử cho thấy hệ thống hoạt động đúng và đáp ứng các yêu cầu đã được thiết kế.

4.4.11 Kết luận

Đồ án đã thực hiện 27 test case quan trọng nhất để kiểm tra các chức năng chính của hệ thống, bao gồm xác thực, quản lý quán ăn, chatbot AI, đánh giá, yêu thích, gợi ý cá nhân hóa, quản trị, giao diện người dùng, và hiệu năng. Các test case này được thực hiện dựa trên yêu cầu chức năng và phi chức năng của hệ thống.

Kết quả kiểm thử cho thấy hệ thống đáp ứng các yêu cầu đã được thiết kế. Các chức năng chính hoạt động đúng, bảo mật được đảm bảo, và hiệu năng hệ thống

nằm trong giới hạn chấp nhận được. Hệ thống đã sẵn sàng cho việc triển khai và sử dụng thực tế.

4.5 Triển khai

4.5.1 Mô hình triển khai

Đồ án được triển khai thử nghiệm trong môi trường phát triển cục bộ với cấu hình sau. Giao diện Người dùng sử dụng máy chủ phát triển Vite chạy trên cổng 5173 với URL `http://localhost:5173` và công cụ đóng gói Vite 5.0+. Phía Máy chủ sử dụng Node.js với Express.js chạy trên cổng 3000 với URL `http://localhost:3000` và trình quản lý quy trình nodemon cho phát triển. Cơ sở Dữ liệu sử dụng PostgreSQL 12+ chạy trên máy chủ localhost, cổng 5432, với tên cơ sở dữ liệu `foodmap_db`.

4.5.2 Cấu hình triển khai

Biến Môi trường trong tệp `.env` bao gồm `DB_HOST=localhost`, `DB_PORT=5432`, `DB_NAME=foodmap_db`, `DB_USER=postgres`, `DB_PASSWORD=<password>`, `JWT_SECRET=<secret_key>`, `GEMINI_API_KEY=<api_key>`, `OPENAI_API_KEY=<api_key>` (tùy chọn), `AI_PROVIDER=auto`, và `CORS_ORIGIN=http://localhost:5173`.

Thiết lập Cơ sở Dữ liệu bao gồm các bước tạo cơ sở dữ liệu bằng lệnh `CREATE DATABASE foodmap_db`, chạy migration bằng lệnh `npm run migrate`, seed dữ liệu mẫu (tùy chọn) bằng lệnh `npm run seed`, và khởi tạo hệ thống học (tùy chọn) bằng lệnh `npm run learn`.

4.5.3 Kết quả triển khai thử nghiệm

4.5.4 Hướng dẫn triển khai production

Để triển khai lên production, đồ án khuyến nghị sử dụng máy chủ Linux (Ubuntu 20.04+) với trình quản lý quy trình PM2 hoặc systemd, reverse proxy Nginx, và SSL/TLS Let's Encrypt. Cơ sở dữ liệu nên được triển khai trên máy chủ riêng hoặc dịch vụ được quản lý với sao lưu định kỳ hàng ngày và cấu hình nhóm kết nối. Giao diện Người dùng nên được đóng gói production bằng lệnh `npm run build`, phục vụ tệp tĩnh từ Nginx hoặc CDN, và bật nén và bộ nhớ đệm. Giám sát nên sử dụng ghi nhật ký với Winston hoặc Pino, theo dõi lỗi với Sentry (tùy chọn), và kiểm tra sức khỏe với điểm cuối `/api/health`.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Trong quá trình thực hiện đồ án tốt nghiệp, ngoài việc hoàn thiện các chức năng cơ bản của một hệ thống quản lý và bản đồ, đồ án đã tập trung nghiên cứu và triển khai một số giải pháp kỹ thuật nhằm nâng cao tính thông minh và trải nghiệm người dùng. Chương 5 này sẽ trình bày chi tiết về ba đóng góp mà đồ án tâm đắc nhất: giải pháp hệ thống tự học từ cơ sở dữ liệu để làm giàu tri thức cho Chatbot, cơ chế gợi ý quán ăn cá nhân hóa dựa trên hồ sơ người dùng, và kiến trúc Chatbot đa mô hình với cơ chế dự phòng.

5.1 Giải pháp hệ thống tự học từ cơ sở dữ liệu

5.1.1 Đặt vấn đề

Một trong những thách thức lớn nhất khi xây dựng Chatbot gợi ý quán ăn là việc cập nhật dữ liệu thực thể. Thông thường, để chatbot nhận diện được tên quán ăn mới hoặc các loại món ăn mới, lập trình viên phải cập nhật thủ công các mẫu câu huấn luyện, điều này gây tốn thời gian và dễ sai sót khi dữ liệu quán ăn thay đổi liên tục. Đồ án đã phát triển một module Hệ thống học từ Database hoạt động tự động khi server khởi động hoặc thông qua lệnh quản trị.

5.1.2 Giải pháp

Đầu tiên, hệ thống có khả năng trích xuất từ khóa tự động bằng cách quét qua toàn bộ các bảng dữ liệu về quán ăn để tự động học các biến thể ngôn ngữ của loại món ăn như Việt Nam, Phở, Bún, tên quán cụ thể, và các địa danh như quận, phường, đường từ địa chỉ thực tế. Thứ hai, dữ liệu mà hệ thống học được sẽ được chuyển đổi thành các mẫu nhận diện thực thể cho bộ lọc xử lý ngôn ngữ tự nhiên. Các mẫu nhận diện thực thể này có thể được cập nhật liên tục xuyên suốt quá trình học. Thứ ba, hệ thống cũng có khả năng tự động nhận diện các tiện ích như wifi, chỗ để xe và các dịp phù hợp như hẹn hò, sinh nhật được ghi nhận trong mô tả quán ăn để dựa vào đó làm cơ sở trong việc gợi ý những quán ăn phù hợp với yêu cầu của khách hàng.

5.1.3 Kết quả đạt được

Hệ thống có khả năng nhận diện ý định và thực thể chính xác hơn mà không cần dữ liệu huấn luyện thủ công. Người quản trị có thể kiểm tra trạng thái học thông qua API `/api/learning/status` và làm mới dữ liệu học ngay lập tức khi có các quán ăn mới được thêm vào.

5.2 Cơ chế gợi ý cá nhân hóa dựa trên hồ sơ người dùng

5.2.1 Đặt vấn đề

Các hệ thống bản đồ thông thường thường gợi ý quán ăn dựa trên khoảng cách địa lý đơn thuần. Tuy nhiên, mỗi người dùng lại có những sở thích, hạn chế ăn uống và ngân sách khác nhau. Việc thiếu đi sự cá nhân hóa khiến các gợi ý đôi khi không thực sự hữu ích.

5.2.2 Giải pháp

Đồ án triển khai hệ thống gợi ý đa tầng tích hợp trực tiếp vào trang RecommendationsPage, bao gồm phân tích hồ sơ, khoảng cách thực tế và lọc dữ liệu thông minh. Đầu tiên, hệ thống trích xuất dữ liệu từ hồ sơ người dùng bao gồm món ăn yêu thích, các loại dị ứng, hạn chế ăn uống và mức giá ưa thích như budget, luxury để lọc các kết quả phù hợp nhất. Tiếp theo, hệ thống sử dụng công thức Haversine để tính toán khoảng cách thực tế từ vị trí Geolocation của người dùng đến tọa độ quán ăn nhằm đảm bảo độ chính xác cho tính năng quán gần đây. Cuối cùng, hệ thống tự động loại bỏ các quán ăn trùng địa chỉ bằng cách chỉ giữ lại quán mới nhất và không gợi ý những quán mà người dùng đã chủ động chặn trong quá khứ.

5.2.3 Kết quả đạt được

Người dùng được cung cấp ba tùy chọn gợi ý rõ ràng bao gồm Dành cho bạn với tính năng cá nhân hóa, Nổi bật dựa trên rating và yêu thích, và Tất cả để xem toàn bộ quán ăn. Giải pháp này giúp tăng tỷ lệ hài lòng của người dùng đối với các đề xuất từ hệ thống.

5.3 Kiến trúc Chatbot đa mô hình với cơ chế dự phòng

5.3.1 Đặt vấn đề

Việc phụ thuộc vào một dịch vụ AI duy nhất như Google Gemini có thể khiến tính năng chatbot của ứng dụng bị tê liệt nếu dịch vụ đó gặp sự cố, bị hết hạn mức quota hoặc gặp lỗi kết nối API.

5.3.2 Giải pháp

Đồ án đã thiết kế một kiến trúc Chatbot linh hoạt với tên gọi Hybrid Chatbot Architecture. Đồ án sử dụng Google Gemini API làm nhà cung cấp chính nhờ khả năng hỗ trợ tiếng Việt tốt và miễn phí quota. Nếu Gemini không khả dụng, hệ thống tự động chuyển đổi sang OpenAI API. Trong trường hợp cả hai API đều không hoạt động, hệ thống sẽ sử dụng bộ xử lý NLP nội bộ thông qua thư viện natural để đưa ra các phản hồi cơ bản. Ngoài ra, hệ thống không chỉ trả lời câu hỏi đơn lẻ mà còn đọc lịch sử từ các phiên đăng nhập trước đó để duy trì mạch hội thoại, tạo cảm giác gần gũi cho người dùng.

5.3.3 Kết quả đạt được

Kiến trúc Chatbot này đảm bảo tính sẵn sàng của chatbot gần như tuyệt đối với khả năng High Availability. Người dùng có thể xóa lịch sử trò chuyện để làm mới phiên làm việc, đảm bảo quyền riêng tư dữ liệu.

Chương này đã làm rõ những giải pháp sáng tạo nhất của đề án, từ việc tự động hóa quá trình học tri thức đến việc cá nhân hóa trải nghiệm và đảm bảo tính ổn định cho hệ thống chatbot. Những đóng góp này không chỉ giải quyết các bài toán kỹ thuật mà còn mang lại giá trị thực tiễn cao cho người dùng. Trong chương tiếp theo, đề án sẽ tổng kết lại toàn bộ kết quả đạt được và định hướng các bước phát triển trong tương lai.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Sau quá trình nghiên cứu, thiết kế và triển khai hệ thống, đồ án đã đạt được những kết quả nhất định trong việc kết hợp công nghệ bản đồ và trí tuệ nhân tạo để giải quyết bài toán tìm kiếm ẩm thực. Chương này sẽ tổng kết các kết quả đạt được, rút ra những bài học kinh nghiệm và đề xuất các hướng phát triển để hoàn thiện sản phẩm trong tương lai.

Đồ án đã hoàn thành mục tiêu xây dựng một hệ thống web ứng dụng tích hợp bản đồ ẩm thực và chatbot AI tư vấn thông minh có tính linh hoạt và dễ mở rộng. Đồ án cũng đã triển khai thành công hệ thống tự học từ cơ sở dữ liệu giúp chatbot tự động cập nhật tri thức mà không cần huấn luyện thủ công. Đồ án tích hợp bản đồ Leaflet tương tác mượt mà với các marker phân loại màu sắc, kết hợp cùng cơ chế gợi ý cá nhân hóa dựa trên hồ sơ, sở thích và vị trí thực tế của người dùng.

So với các ứng dụng bản đồ thông thường, sản phẩm mang lại sự tương tác tự nhiên hơn thông qua chatbot hỗ trợ cả hai mô hình AI hàng đầu là Google Gemini và OpenAI. Quá trình thực hiện đồ án cũng giúp tích lũy được nhiều bài học quý giá về việc xử lý ngôn ngữ tự nhiên tiếng Việt, quản lý dự án Full-stack và tối ưu hóa hiệu năng ứng dụng web.

6.2 Hướng phát triển

Mặc dù đã đạt được những mục tiêu cơ bản, hệ thống vẫn còn nhiều dư địa để cải thiện và nâng cấp nhằm phục vụ người dùng tốt hơn.

Một mặt, đồ án cần nâng cao khả năng xử lý ngôn ngữ để chatbot hiểu sâu hơn các câu lệnh phức tạp. Mặt khác, hệ thống gợi ý có thể được tối ưu tốt hơn để dự đoán chính xác hơn xu hướng ăn uống của người dùng dựa trên hành vi cộng đồng. Hơn nữa, đồ án cũng có thể mở rộng tính năng như tích hợp hệ thống thông báo thời gian thực và phân tích cảm xúc từ các đánh giá của người dùng để xếp hạng quán ăn khách quan hơn. Ngoài ra, việc phát triển phiên bản ứng dụng di động để người dùng có thể sử dụng hệ thống thuận tiện hơn khi đang di chuyển cũng là một hướng đi đồ án có thể cải thiện. Cuối cùng, để kiểm soát ứng dụng tốt hơn, cần phải xây dựng một bảng điều khiển giúp quản trị viên theo dõi những thông tin quan trọng như lưu lượng truy cập, thống kê các từ khóa được tìm kiếm nhiều nhất và xuất báo cáo dữ liệu định kỳ.

Việc tiếp tục phát triển theo các hướng này không chỉ giúp hoàn thiện sản phẩm về mặt kỹ thuật mà còn mở ra cơ hội ứng dụng thực tiễn rộng rãi trong lĩnh vực

dịch vụ và du lịch.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

PHỤ LỤC